



UNIVERSIDAD ALFONSO X EL SABIO

Business Tech

Máster Universitario en Inteligencia Artificial

TRABAJO DE FIN DE MÁSTER

Late Fusion of Transformer-Based
Financial Sentiment and Technical Analysis
via Reinforcement Learning for Equity Trading

Sergio Nicolás Seguí

Director: Francisco José Soltero Domingo

Curso académico 2025–2026

Abstract

This thesis investigates whether combining two heterogeneous information sources — quantitative price signals and qualitative financial-news sentiment — through an explicit **late-fusion** architecture provides measurable benefit over an **early-fusion** baseline in a Reinforcement-Learning-based equity trading system. The system is composed of three modular components: (i) a sentiment module built on the Transformer-based FinBERT model, which converts news articles into a daily (score, confidence) pair per ticker; (ii) a technical module based on XGBoost over 15 hand-engineered price features, producing an analogous (score, confidence) pair; and (iii) a PPO agent that learns to translate these four numbers, together with three lagged returns, into trading decisions under a risk-adjusted reward.

We evaluate the system on ten large-cap S&P 500 constituents over an explicit time-leakage-free pipeline. Training covers 2018–2022, validation 2023, and test 2024–2025 (496 trading days, 4,760 ticker-days). After a documented diagnostic iteration that removed the short action and replaced the Sharpe reward with the Sortino reward, the final late-fusion agent achieves a Sharpe of **+1.11**, a Sortino of **+1.77**, a maximum drawdown of only **−6.0%**, and a Calmar ratio of **2.39** — the best risk-adjusted profile in the ablation. Late fusion does **not** exceed the cumulative return of buy-and-hold (+30.2% vs +121.7%) during a strongly bullish test window, but it reduces drawdown by approximately **4×** and delivers worst-month losses of **−2.2%** versus **−10.2%** for the passive benchmark.

Auxiliary analyses confirm the result is **not** driven by one or two tickers, quantify the strategy’s transaction-cost sensitivity, and document the agent’s position decisions across market regimes. The work makes three contributions: an explicit (score, confidence) abstraction that enables independent module evaluation; an honest experimental record of how reward and action-space design shape out-of-sample behaviour; and a reproducible open-source pipeline.

Keywords: reinforcement learning, FinBERT, Transformer, late fusion, algorithmic trading, PPO, XGBoost, sentiment analysis, risk-adjusted return, backtesting.

Resumen

Este trabajo investiga si combinar dos fuentes heterogéneas de información — señales cuantitativas de precios y sentimiento cualitativo de noticias financieras — mediante una arquitectura explícita de **fusión tardía** aporta un beneficio medible frente a la **fusión temprana** habitual en sistemas de *trading* basados en aprendizaje por refuerzo. El sistema se compone de tres módulos: (i) un módulo de sentimiento basado en el modelo Transformer especializado FinBERT que transforma noticias en un par (*score*, confianza) diario por valor; (ii) un módulo técnico basado en XGBoost sobre 15 indicadores de precio que produce un par análogo; y (iii) un agente PPO que aprende a convertir esas cuatro magnitudes, junto con tres rendimientos rezagados, en decisiones de *trading* bajo una recompensa ajustada por riesgo.

El sistema se evalúa sobre 10 valores *large-cap* del S&P 500 con un protocolo estricto de prevención de fugas temporales. El entrenamiento cubre 2018–2022, la validación 2023, y el test 2024–2025 (496 días, 4.760 *ticker*-días). Tras una iteración diagnóstica documentada que elimina la acción de venta corta y sustituye la recompensa de Sharpe por Sortino, el agente final de fusión tardía logra un Sharpe de **+1,11**, un Sortino de **+1,77**, un *drawdown* máximo de solo **−6,0%** y un Calmar de **2,39** — el mejor perfil ajustado por riesgo del estudio. La fusión tardía **no** supera el retorno acumulado del *buy-and-hold* (+30,2% vs +121,7%) durante una ventana de test fuertemente alcista, pero reduce el *drawdown* aproximadamente **4×** y limita la pérdida del peor mes a **−2,2%** frente al **−10,2%** del benchmark pasivo.

Los análisis auxiliares confirman que el resultado no depende de uno o dos valores, cuantifican la sensibilidad a comisiones y documentan las decisiones del agente bajo distintos regímenes de mercado. Las tres contribuciones principales son: una abstracción explícita (*score*, confianza) que permite evaluar cada módulo de forma independiente; un registro experimental honesto de cómo el diseño de recompensa y espacio de acción condicionan el comportamiento fuera de muestra; y una *pipeline* reproducible de código abierto.

Palabras clave: aprendizaje por refuerzo, FinBERT, Transformer, fusión tardía, trading algorítmico, PPO, XGBoost, análisis de sentimiento, retorno ajustado por riesgo, *back-testing*.

Academic and Financial Disclaimer

This document is an **academic Master's thesis**. All results are obtained from **historical backtests** on a fixed universe of ten large-cap United States equities over the period 2018–2025. **Past performance is not indicative of future results.** The strategies, models and code described here are research artefacts: they have not been deployed in a real brokerage account, they ignore several frictions present in real markets (slippage variation across asset liquidity, partial fills, borrow costs for short positions, intra-day execution risk), and they have been trained on a limited universe and time window.

Nothing in this document constitutes financial, investment, legal or tax advice, nor an offer or solicitation to buy or sell any security. Any reader who chooses to act on the ideas described here does so entirely at their own risk and should consult a qualified financial professional first.

Table of Contents

1	Introduction	8
1.1	Context and Motivation	8
1.2	A Note on Terminology: FinBERT, LLMs and Transformers	9
1.3	The Efficient Market Hypothesis and Its Implications	10
1.3.1	The Three Forms of Efficiency	11
1.3.2	Why the EMH Does Not Forbid Predictability	11
1.3.3	Implications for This Work	12
1.4	Problem Formulation	12
1.4.1	Why a Markov Decision Process?	12
1.4.2	MDP Specification	13
1.4.3	Where the Markov Property Fails — and Why It Still Works	14
1.5	The Late-Fusion Hypothesis	14
1.5.1	Early Fusion: the Default Choice	14
1.5.2	Late Fusion: the Alternative	15
1.5.3	Three Concrete Benefits	15
1.6	Objectives	16
1.7	Document Structure	16
2	Background and Related Work	18
2.1	Foundations of Reinforcement Learning	18
2.1.1	The Reinforcement Learning Framework	18
2.1.2	Exploration vs Exploitation	19
2.1.3	Value Functions and the Bellman Equations	19
2.1.4	Q-Learning and Deep Q-Networks	19
2.1.5	Policy Gradient Methods	20
2.1.6	Proximal Policy Optimisation	20
2.1.7	The Soft Actor-Critic Alternative	22
2.2	Reinforcement Learning for Algorithmic Trading	22
2.2.1	Historical Development	22
2.2.2	Key Challenges in Financial RL	22
2.2.3	Reward Function Design	23
2.3	Transformer-Based Language Models	23
2.3.1	The Transformer Architecture	23
2.3.2	BERT: Pre-training and Fine-tuning	24
2.3.3	FinBERT: Domain Adaptation for Financial Text	26
2.4	LLMs for Financial Sentiment and Forecasting	28
2.5	Integration of LLMs and Reinforcement Learning for Trading	28
2.5.1	Early-Fusion Approaches	28
2.5.2	Gap in the Literature	28
3	System Design and Implementation	29
3.1	Overall Architecture	29
3.2	Data Collection	29
3.2.1	Equity Price Data	29

3.2.2	Financial News Data and Temporal Coverage	30
3.3	Temporal Alignment and Leakage Prevention	31
3.3.1	The Leakage Problem	31
3.3.2	Alignment Rules	32
3.3.3	Worked Example	32
3.3.4	Automated Verification	32
3.4	Dataset Splits	33
3.5	Sentiment Analysis Module	33
3.5.1	Architecture and Model Selection	33
3.5.2	Input and Score Computation	33
3.5.3	Daily Aggregation	33
3.5.4	Standalone Evaluation on Financial PhraseBank	35
3.5.5	Predictive Signal Validation	35
3.5.6	Score Distribution Across Splits	36
3.6	Technical Analysis Module	36
3.6.1	Feature Engineering	36
3.6.2	XGBoost Classifier	36
3.6.3	Hyperparameter Tuning	36
3.6.4	Why the Technical Score Is So Weak	37
3.6.5	Score Construction	37
3.7	Trading Environment	37
3.7.1	Gym Interface	37
3.7.2	State Space	38
3.7.3	Action Space (v1 vs v2)	39
3.7.4	Transaction Cost	39
3.7.5	Episode Structure	39
3.7.6	Reward Function	40
3.7.7	Environment Validation	41
3.8	Fusion Model	41
3.8.1	Architecture	41
3.8.2	Base Training Configuration	42
3.8.3	Hyperparameter Tuning	42
4	Methodology and Experimental Design	43
4.1	Why an Ablation Study	43
4.2	Ablation Study Protocol	43
4.3	Freeze Protocol and Honest Disclosure on v1 $\rightarrow$ v2	44
4.4	Evaluation Metrics	44
4.5	From v1 to v2: Diagnostic Iteration	46
4.5.1	Diagnostic Summary	46
4.5.2	Modifications for v2	47
4.5.3	Side Effects of the Sortino Reward (Brief)	47
4.6	Dynamic Routing: Methodology	47
5	Results	48
5.1	Summary Table of All Configurations (Test Split)	48

5.2	Standalone Module Behaviour and the B2 Threshold Bug	51
5.3	v1 Ablation — Diagnostic Result	51
5.4	v2 Ablation — Post-Diagnostic Results	52
5.5	Per-Asset Analysis	52
5.6	Per-Regime Performance and Position Decisions	54
5.6.1	Regime-Conditional Confidence (Inputs)	56
5.7	Transaction-Cost Sensitivity	56
5.8	Robustness — Late Fusion vs Buy-and-Hold (Monthly)	58
5.9	Dollar Backtests — How Much Money Would It Make?	59
5.10	Action Distribution and Behavioural Footprint	60
5.11	Regime $\times$ Position — Dynamic Routing Re-examined	61
6	Discussion	62
6.1	Economic Interpretation: Defensive Strategy, Not Alpha Generator	62
6.1.1	Utility-Theoretic Framing	62
6.1.2	Practical Deployment Profiles	62
6.1.3	The Strategy as a Risk-Management Tool	63
6.2	Late Fusion vs Early Fusion — Why the Gap Is Real	63
6.2.1	Quantitative Magnitude of the Gap	64
6.2.2	What Could Explain the Gap	64
6.2.3	Why the Implicit Discovery Fails	64
6.2.4	The Architectural Lesson	65
6.3	Side Effects of the Sortino Reward	65
6.4	The Role of Confidence as an Architectural Bottleneck	66
6.5	A Note on Technical Analysis “Not Working”	66
6.6	Comparison with Related Work	67
7	Conclusions	68
7.1	Final Statement	69
8	Limitations and Threats to Validity	71
9	Future Work	74
	References	76

1 Introduction

1.1 Context and Motivation

Financial markets are among the most complex adaptive systems studied in the social sciences. Unlike physical systems governed by deterministic laws, markets are the emergent result of the simultaneous actions of millions of heterogeneous agents — retail investors, institutional funds, algorithmic traders, central banks — each acting on incomplete information and anticipating the behaviour of others. This self-referential structure makes prediction fundamentally difficult: any consistently profitable strategy tends to attract imitators, eroding the edge it was based on. The classic illustration is the parable of the **hundred-dollar bill on the sidewalk**: an efficient-markets economist refuses to believe it is real because “someone would have picked it up by now.” Real markets occupy an intermediate position — bills **do** appear, but the half-life of any visible bill is short, and detecting one requires infrastructure that itself costs money. Persistent profitability is therefore a question not of “is there any pattern?” but of “can a pattern be detected and exploited **faster than the competition** with positive net economics after costs?”

Despite this adversarial nature — or perhaps because of it — automated trading systems have become a dominant force in modern markets. By regulatory estimates, high-frequency trading firms account for over 50% of daily volume in major US equity markets, and quantitative hedge funds (Renaissance Technologies, Two Sigma, D.E. Shaw, Citadel, AQR) collectively manage trillions of dollars using systematic, data-driven strategies. The pioneering work was largely **price-only**: statistical arbitrage on historical price relationships, futures basis trades, index arbitrage, latency arbitrage. As these traditional sources of edge were progressively arbitrated away — bid-ask spreads collapsing from fractions of a dollar to fractions of a cent, latency battles measured in nanoseconds — the frontier moved to **alternative data**: satellite imagery of parking lots, credit-card spending panels, web-scraped product reviews, shipping-container manifests, and most importantly for this thesis, large-scale ingestion of textual content (news, filings, social media, earnings transcripts).

The question is therefore no longer whether machines can trade — they demonstrably can and do at industrial scale — but **what information they should use** and **how they should combine it**. Two fundamentally different types of information drive price movements. The first is **quantitative**: prices, volumes, order-book depth, and derived technical indicators such as moving averages, momentum measures, mean-reversion oscillators, and volatility estimators. This information is structured, numerical, available in real time, easily ingested by classical machine learning models, and — most importantly — **symmetric across all participants**. Every trader sees the same price tape; any edge from processing it has to come from speed, scale, or novel transformations of public data.

The second category is **qualitative**: news articles, earnings call transcripts, regulatory filings (10-K, 10-Q, 8-K), social media posts, analyst reports, central bank communications, and macroeconomic announcements. This information is unstructured, often ambiguous,

frequently contradictory across sources, and requires **language understanding** to process correctly. The same press release can be parsed as bullish by a naive bag-of-words classifier (“revenue grew”, “strong performance”) and as bearish by a competent reader who notices “guidance lowered” three paragraphs later. Historically, qualitative signals have been the domain of human analysts; the obstacle to their systematic use was that no machine could read financial text well enough at scale to compete with humans on the inputs they actually use.

The emergence of Transformer-based language models, beginning with BERT [1] in 2018 and continuing through the LLM revolution of 2020–2024, has fundamentally changed this picture. For the first time, qualitative signals are accessible to automated systems at the same throughput and quality as quantitative ones. A retail-tier GPU can run FinBERT inference on tens of thousands of news articles per hour at near-human classification accuracy. The **technological precondition** for systematic exploitation of qualitative signals is now satisfied; the open question is the **architectural one**: how should a trading system **combine** quantitative and qualitative inputs, given that they have radically different semantic structures, update frequencies, noise characteristics, and confidence calibration?

The integration of these two signal types into a unified trading system is the central challenge addressed by this work. The approach taken here is deliberately modular: rather than building a single monolithic model that processes all information jointly, we design two independent modules — one for quantitative technical analysis and one for qualitative sentiment analysis — and learn a fusion policy that combines their outputs through reinforcement learning. The choice of modularity has both **scientific** motivations (independent diagnosability, isolated evaluation of each component, interpretable failure attribution) and **practical** motivations (the two modules can be developed, versioned, and replaced independently; a failure in the sentiment pipeline does not invalidate the technical pipeline; the system as a whole can be debugged component-by-component rather than as an opaque black box). The trade-off is that the fusion model loses access to fine-grained cross-modal interactions that a monolithic model might exploit. Whether the gains from modularity outweigh this loss is the central empirical question of the thesis, answered through a controlled ablation study comparing late fusion (modular) against early fusion (joint) under identical conditions.

1.2 A Note on Terminology: FinBERT, LLMs and Transformers

A genuine terminological ambiguity must be addressed at the outset of this work, particularly because an earlier draft used the looser term **LLM sentiment** in the title. Modern usage of the term “Large Language Model” has shifted significantly over time, with three overlapping definitions in active use across academia and industry:

1. **Original (pre-2020) usage**: any neural language model with hundreds of millions of parameters trained on large text corpora — under this definition, BERT (110M parameters), GPT-2 (1.5B), T5 (11B), and FinBERT all qualify.

2. **Transitional (2020–2022) usage:** language models of approximately 1 billion parameters or more, regardless of architecture (encoder, decoder, encoder-decoder) — BERT no longer clearly qualifies, but T5-large, GPT-3 (175B), and BLOOM (176B) do.
3. **Contemporary (2023–present) usage:** large **autoregressive generative** models capable of in-context learning, instruction following, and emergent reasoning — typically 7B parameters or more (Llama-3, GPT-4, Claude, Gemini). Under this strict definition, **all encoder-only models** (BERT, RoBERTa, FinBERT) are **excluded** from the LLM category regardless of size.

FinBERT [2] is a BERT-base encoder of approximately 110 million parameters, fine-tuned for three-class classification rather than generative text completion. Under definition (1) it is an LLM; under (2) it is borderline; under the modern strict definition (3) it is **not** an LLM but rather a **Transformer-based specialised language model** or, more precisely, a **domain-adapted bidirectional Transformer encoder for financial sentiment classification**.

The earlier working title of this thesis used “LLM sentiment” in the loose sense (1), emphasising the architectural family (Transformer, large-corpus pre-training, learned semantic representations) and the contrast against classical dictionary-based or bag-of-words sentiment baselines such as Loughran-McDonald [3]. To avoid confusion with the modern strict sense — under which a reader might reasonably expect a system using GPT-4 or Claude — the title has been revised to **Transformer-Based Financial Sentiment**, and the body of the text consistently refers to FinBERT as a **Transformer-based encoder**.

This terminological precision matters substantively, not just nominally. The properties typically attributed to **modern** LLMs — in-context learning, chain-of-thought reasoning, instruction following, the ability to ingest arbitrary new task descriptions at inference time — are **not** properties of FinBERT. What FinBERT **does** provide is robust, fast, batch-friendly classification with calibrated probabilities, at three orders of magnitude lower inference cost than a 7B-parameter generative model. The choice is an explicit trade-off: worse semantic depth and reasoning, but better throughput, cost, and deterministic behaviour. We revisit this trade-off in Sections 2.4 and 6.4, and list the choice as an explicit limitation in Chapter 8 where we propose generative-LLM replacements as a future-work direction.

1.3 The Efficient Market Hypothesis and Its Implications

The Efficient Market Hypothesis (EMH) [4] is the foundational theoretical framework for understanding what algorithmic trading systems can and cannot achieve. It is not a falsifiable scientific theory in the strict Popperian sense — Fama himself noted that EMH must be tested **jointly** with a specific asset-pricing model, a methodological observation known as the **joint-hypothesis problem**. Despite this, EMH is the right intellectual scaffolding for any honest discussion of algorithmic trading because it correctly predicts

both the **order of magnitude** and the **fragility** of the alpha that any real strategy can extract from public information.

1.3.1 The Three Forms of Efficiency

Fama's taxonomy distinguishes three nested forms of market efficiency:

Weak form. Current prices fully reflect all **historical price and volume** information. The implication is that **pure technical analysis** — strategies based solely on past prices, volumes, and derived indicators — cannot generate consistent excess returns after costs. This is the regime in which our technical module operates: it consumes only OHLCV data and learns a binary predictor of 5-day forward direction.

Semi-strong form. Current prices reflect all **publicly available information** — financial statements, news, analyst reports, regulatory filings, macroeconomic releases. Under this hypothesis, even a perfect news-reading system cannot systematically outperform the market, because every participant has access to the same news and the market price is the equilibrium aggregation of their interpretations. This is the regime in which our sentiment module operates: it processes financial news that any participant could read.

Strong form. Current prices reflect **all** information including private (insider) information. This form is widely regarded as empirically false — SEC enforcement actions against insider trading are direct evidence that insider information **does** generate excess returns when acted upon. Strong-form efficiency is not relevant to our work because we do not use any private data.

1.3.2 Why the EMH Does Not Forbid Predictability

A frequent misreading of the EMH is that it forbids **any** exploitable pattern in prices. This is not what it says. The precise claim is that, after adjusting for **risk** and **costs**, no pattern can yield economically significant excess returns **consistently and at scale**. This leaves substantial room for predictability of the following kinds:

1. **Sub-cost autocorrelations.** Short-term price autocorrelations of magnitude too small to overcome bid-ask spreads exist routinely. The market is efficient in the sense that nobody can trade them profitably; it is not efficient in the sense that they are absent.
2. **Risk-premium effects.** Strategies that systematically take on uncompensated-by-the-market risks (volatility, illiquidity, regime change) can earn premia that, properly viewed, are not “alpha” but compensation for risk-bearing.
3. **Information-processing speed.** If new public information takes hours to fully impound into prices, a participant who reads and acts faster captures a transient edge. This is the regime exploited by news-driven trading.
4. **Behavioural anomalies.** Some patterns persist for non-rational reasons — end-of-month rebalancing flows, retail option-expiration dynamics, post-earnings-announcement drift [5] — and are stable enough that they survive in measurable form.

1.3.3 Implications for This Work

The EMH framework predicts three things about our experimental setup that we indeed observe:

1. **The technical module should be weak.** On large-cap, heavily-followed US equities with a 5-day binary forward horizon and a textbook indicator set, any signal that an XGBoost classifier could discover should already have been arbitrated out by the systematic players that dominate volume in these names. Our validation AUC of 0.5188 — barely above random chance — confirms this. We discuss the diagnostic implications in Section 3.6.4.
2. **The sentiment module should be weak but not zero.** Public news is observable by everyone, but the **interpretation** of news has costs (cognitive effort, inference latency, attention bandwidth) that not all participants pay in equal measure. A Transformer-based system that reads news faster and more consistently than the marginal participant can capture a small, persistent edge. We measure this edge as Pearson $r = +0.036$ ($p = 0.0014$) on the training split — small, statistically significant, and consistent with the semi-strong efficiency reading.
3. **The fusion model can only multiply small edges.** Both individual modules carry modest signal, so even an architecturally optimal fusion cannot turn $0.5 + 0.04 = 0.54$ accuracy into 0.8 accuracy. What fusion **can** do is improve the **risk-adjusted** deployment of the available signal: it can hold positions longer in regimes where both modules agree, exit positions faster when they disagree, and use module confidence as an implicit position-sizing signal. The ablation results in Chapter 5 are consistent with exactly this reading: late fusion does not dramatically improve raw Sharpe over the sentiment-only baseline, but it dramatically improves drawdown control.

Both findings together — weak technical signal, modest sentiment signal, significant risk-adjusted improvement from fusion — paint a coherent picture that the EMH framework would have predicted **qualitatively** without any experiments. The contribution of this work is therefore **not** a falsification of EMH (which would be very strong indeed) but a **quantification** of how much risk-adjusted improvement is achievable from a specific architectural choice under a strictly leakage-free evaluation protocol.

1.4 Problem Formulation

1.4.1 Why a Markov Decision Process?

We formulate the trading problem as a discrete-time **Markov Decision Process** (MDP) rather than as a **supervised learning** problem for three reasons that are central to the architecture of the rest of this work.

Reason 1: actions affect future state. In a supervised regression of next-day return on current features, the model implicitly assumes that its prediction does not influence subsequent feature values. This is **true** for single-shot signal extraction (the sentiment module, the technical module) but **false** for the trading decision itself: if the agent enters a

position today, it must observe the consequences (P&L, change in inventory, transaction cost already incurred) before making tomorrow's decision. The trading decision is **path-dependent** on its own previous decisions, which is the canonical signature of a sequential decision problem.

Reason 2: reward is delayed and risk-adjusted. The “ground truth” for a trading decision today is not a single label observable tomorrow — it is a distribution of P&L outcomes over a multi-day forward window, jointly with the volatility and drawdown profile of those outcomes. A supervised loss can be constructed for the mean return, but constructing a supervised loss for the risk-adjusted return requires explicitly enumerating the forward window and its statistics. RL naturally accommodates such window-based reward design.

Reason 3: exploration is required. Any single ticker-year of historical data contains only one realised trajectory of prices. The agent must learn from **one** realisation what would have happened under **counterfactual** actions. Supervised learning can side-step this by labelling each day with its realised reward under the hypothetical action “always long”, but this loses the path-dependence and forces a separate model per discrete action. RL learns the joint policy directly.

1.4.2 MDP Specification

Formally, our MDP is the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ with:

- $\mathcal{S} = [-1, 1]^7$ — a 7-dimensional continuous hypercube. The seven dimensions are $(s_{\text{sent}}, c_{\text{sent}}, s_{\text{tech}}, c_{\text{tech}}, \tilde{r}_{t-1}, \tilde{r}_{t-2}, \tilde{r}_{t-3})$ for sentiment and technical (score, confidence) and three lagged normalised log-returns. We discuss the rationale for this specific state design in Section 3.7.2.
- $\mathcal{A}$ — discrete action space. In v1 of the environment $\mathcal{A} = \{-1, 0, +1\}$ (sell/hold/buy); in v2 $\mathcal{A} = \{0, +1\}$ (cash/long). The reasons for this revision are documented in Section 4.4.
- $P(s' | s, a)$ — the transition dynamics. In our setting the dynamics are **exogenous**: the next state is determined by tomorrow's market data (next prices, next news), not by the agent's action. The agent affects only its internal position and accumulated P&L, not the market itself. This is the **price-taker** assumption: appropriate for retail and mid-size institutional trading on liquid mega-cap names, where individual orders do not move prices. It would be inappropriate for HFT or block trading where market impact matters.
- $R(s, a, s')$ — the reward function. Discussed in Section 3.7.5; the final v2 form uses a clipped 5-day forward Sortino ratio penalised by transaction cost.
- $\gamma \in [0, 1)$ — the discount factor. We use $\gamma = 0.9587$ (Optuna-tuned). This means a reward 22 trading days (1 month) in the future is discounted by approximately $0.9587^{22} \approx 0.40$. Shorter discounting is appropriate because (i) the reward function already uses a 5-day forward window so longer-horizon rewards are partially redundant, (ii) financial regime change makes very-long-horizon credit assignment unreliable.

The agent's objective is the standard expected discounted return:

$$J(\pi) = \mathbb{E}_{\pi} \left[\sum_{t=0}^T \gamma^t R_t \right]$$

1.4.3 Where the Markov Property Fails — and Why It Still Works

Strictly, the Markov property — “the future is conditionally independent of the past given the present” — **fails** in financial markets. Returns have memory: volatility clusters, momentum persists for weeks, mean-reversion operates over months. A truly Markovian state would need to include all of these statistics explicitly.

Our state representation is therefore an **approximation**: we include three lagged returns to capture immediate momentum, and we **bet** that the longer- horizon memory is implicitly captured through the price-derived technical indicators (EMAs, Bollinger Bands, RSI) inside the technical module’s input features. The technical score s_{tech}^t is a learned summary of 15 features that themselves span lookback windows of 5, 10, 14, 20, and 50 days. This is **not** a perfect Markov state, but it is an empirically adequate one — the agent’s behaviour does not deteriorate over long episodes, which it would if the missing-state assumption were severely violated.

The lesson, generalisable to any RL problem on non-Markovian dynamics, is that the state representation must compress **the past** into a statistic **sufficient for the future**. Our design pushes this compression into the upstream modules (FinBERT for text, XGBoost for prices) and lets the RL agent focus on the much lower-dimensional **signal interpretation** problem.

1.5 The Late-Fusion Hypothesis

The central contribution of this work is the **late-fusion architecture**. The distinction between **early** and **late** fusion is well-established in multi- modal machine learning (image+text, audio+video, sensor fusion in robotics) but is comparatively under-explored in financial RL, where early fusion dominates.

1.5.1 Early Fusion: the Default Choice

In the dominant **early-fusion** approach, text signals are converted to numerical features and concatenated with price features to form a single joint state vector that is fed directly to an RL agent [6]. The text features might be the full hidden-state vector from a Transformer (768 dimensions), the pooled classification embedding, the softmax over classes, or simple dictionary counts. Whatever the choice, the philosophical commitment is that **the RL policy network will learn the appropriate combination of all input dimensions jointly from the reward signal**.

This approach has three undeniable advantages: (i) implementation simplicity — concatenate-and-train; (ii) the policy network can in principle discover **arbitrarily complex** cross-modal interactions that a designer would not think to specify; (iii) no information is lost in the upstream summarisation.

It also has three disadvantages, all of which motivate the late-fusion alternative:

1. **Diagnosability collapses.** When the early-fusion agent underperforms, the failure mode is opaque. Did the price features dominate the gradient and drown out the text signal? Did the text features overfit on training patterns that did not generalise? Was the action choice wrong even when the features were right? With a single monolithic state vector and a single policy network, attribution requires sophisticated post-hoc interpretability tools (SHAP, attention rollout, ablation by zeroing dimensions) that are themselves unreliable on small RL training sets.
2. **Module evaluation is impossible.** The text encoder and the price-feature pipeline cannot be evaluated independently against domain-specific benchmarks. FinBERT’s classification accuracy on Financial PhraseBank is **not measured** by the trading reward — it is **implied** by the trading reward via a long causal chain. A modeller cannot say “the sentiment pipeline is at 87.6% F1, let me work on the technical pipeline next” because there is no isolated “sentiment pipeline” to measure.
3. **Cross-modal routing is not exposed.** In real markets, the appropriate weight on news sentiment varies across regimes: it should be high when news is dense and confidently directional, and low when news is sparse or contradictory. An early-fusion agent **might** learn this routing implicitly, but the modeller cannot inspect or verify that it has done so.

1.5.2 Late Fusion: the Alternative

Late fusion, by contrast, treats each information source as an independent **expert** that produces a **standardised output**: a (score, confidence) pair, both bounded in $[-1, 1]$ and $[0, 1]$ respectively. The **score** encodes the **direction** of the signal — positive for bullish, negative for bearish, near zero for ambiguous. The **confidence** encodes **how strongly the module commits to its direction**, with a clear semantic meaning that we preserve throughout the system: zero confidence means “I have nothing to contribute”; high confidence means “trust this score”.

The pair $(s, c) \in [-1, 1] \times [0, 1]$ is therefore a **compressed, standardised, interpretable** representation of an expert’s opinion. A separate **fusion model** — in our case a PPO agent — learns to aggregate these pairs into trading decisions. The fusion model receives a **homogeneous** input regardless of whether the signal originated as text or as price, and it can learn a **generic** signal-quality gating rule (e.g., “take a position proportional to $|s| \cdot c$, sign with $\text{sgn}(s)$ ”) that transfers across modalities.

1.5.3 Three Concrete Benefits

The design enables three concrete benefits that we exploit throughout the thesis:

Independent evaluation. Each module can be benchmarked against domain-specific tasks **before** integration. FinBERT is evaluated on Financial PhraseBank against the published $F1 = 0.876$ (Section 3.5.5). The technical XGBoost module is evaluated on validation AUC (Section 3.6.3). If either module is broken, it is detected at the module level, not the system level.

Interpretability. The fusion model’s behaviour can be analysed by **correlating its actions with module confidence levels**. We do exactly this in Section 5.7 and find a $2.2\times$ variation in long-share across regimes — direct evidence that the agent is performing signal-quality gating.

Dynamic routing. The fusion model can, in principle, learn to **up-weight** the sentiment module when news coverage is dense and confidently directional, and **down-weight** it when news is sparse or ambiguous (defaulting to the technical signal and recent returns). We call this the **dynamic-routing hypothesis** and design the ablation study and regime analysis to test it empirically.

The trade-off, made explicit, is that **cross-modal information is lost** in the compression to two scalars. A monolithic model that sees the full FinBERT hidden state and the full 15-dim XGBoost feature vector could in principle exploit fine-grained interactions (e.g., “bullish sentiment in `bb_pct` > 0.8 regime”) that our compressed representation discards. We are betting that the gain from independent diagnosability, the small training budget (12,590 days), and the explicit confidence channel outweigh the loss from compression. The B4 vs P ablation directly tests this bet.

1.6 Objectives

1. Design and implement a **sentiment module** based on FinBERT that produces a structured (score, confidence) pair per trading day per asset, with a rigorous temporal-alignment protocol that prevents data leakage.
2. Design and implement a **technical module** based on XGBoost over 15 price and volume indicators, producing an analogous (score, confidence) pair.
3. Design and implement a **PPO-based fusion model** that learns to combine both (score, confidence) pairs into trading actions under a risk-adjusted reward.
4. Evaluate the full system through a controlled **ablation study** comparing five configurations on identical splits with a strictly leakage-free pipeline.
5. Investigate the **dynamic-routing hypothesis** by analysing both the agent’s input confidences and its actual position decisions across market regimes.

1.7 Document Structure

Chapter 2 provides the theoretical background on RL, Transformer-based language models and their application to financial trading. Chapter 3 describes the system design and the data pipeline. Chapter 4 describes the experimental design, including the diagnostic iteration from v1 to v2. Chapter 5 reports all empirical results, including per-asset, per-regime and cost-sensitivity analyses. Chapter 6 discusses the economic interpretation of the results, side-effects of the Sortino reward, and architectural comparison with related

work. Chapter 7 presents conclusions tied directly to the objectives. Chapters 8 and 9 list the work's limitations and propose future extensions.

2 Background and Related Work

2.1 Foundations of Reinforcement Learning

2.1.1 The Reinforcement Learning Framework

Reinforcement learning is a computational framework for learning decision-making policies through interaction with an environment [7]. Unlike supervised learning, where the training signal is a **labelled correct output**, the RL training signal is a **scalar reward** received **after** the action, often **delayed** by many steps, and dependent on the **agent's own behaviour** through the trajectory of visited states. This combination of delayed, sparse, action-dependent feedback is what makes RL both powerful (it requires no labelled data) and difficult (credit assignment over long horizons is unstable).

The interaction is modelled as a **Markov Decision Process** (MDP), defined by a tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ where:

- $\mathcal{S}$ is the **state space** — the set of all possible observations the agent can receive. It can be discrete (e.g., positions on a chessboard) or continuous (e.g., joint angles of a robot, or the 7-dim hypercube used in this work).
- $\mathcal{A}$ is the **action space** — the set of all possible actions the agent can take. It can be discrete (Atari games, our trading actions) or continuous (steering angles, portfolio weights).
- $P : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S})$ is the **transition dynamics** — a probability distribution over next states given the current state and action. In finance, P is **exogenous** in the price-taker limit: the market evolves independently of any single small participant's actions.
- $R : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is the **reward function** — the immediate signal received after an action. Design of R is one of the most consequential decisions in any RL system; Section 4.4.3 documents how our specific choice (Sortino vs Sharpe) shapes the resulting policy.
- $\gamma \in [0, 1)$ is the **discount factor** — the relative weight of immediate vs future rewards. $\gamma = 0$ produces a myopic agent that optimises one step ahead; $\gamma \rightarrow 1$ produces an agent that values all future rewards equally.

The interaction unfolds in discrete time: at each step t , the agent observes $s_t \in \mathcal{S}$, samples an action $a_t \sim \pi(\cdot | s_t)$ from its **policy** π , receives a reward $r_t = R(s_t, a_t)$, and transitions to $s_{t+1} \sim P(\cdot | s_t, a_t)$. The objective is to find a policy that maximises the **expected discounted return**:

$$J(\pi) = \mathbb{E}_{\pi}[G_t] = \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \right]$$

The Markov property — **the future is conditionally independent of the past given the present** — is central to the MDP formalism. In financial trading this assumption is **violated** in general (markets have memory through volatility clustering, momentum, and regime persistence), but it provides a useful approximation if the state representation is rich enough to capture the relevant history. The standard remedy is to include lagged

observations in s_t — which is exactly the role of the three lagged returns $\tilde{r}_{t-1}, \tilde{r}_{t-2}, \tilde{r}_{t-3}$ in our state design.

2.1.2 Exploration vs Exploitation

The defining tension in RL is between **exploiting** known-good actions and **exploring** unknown actions that might be even better. The exploitation-only agent would always take the action with the highest currently estimated value, but if the estimate is wrong the agent never discovers its error. The exploration-only agent would sample uniformly across actions and learn the value function, but never reap the benefits.

Practical RL algorithms balance the two with mechanisms such as ϵ -greedy exploration (act greedily with probability $1 - \epsilon$, randomly with probability ϵ), Boltzmann exploration (sample from a softmax over estimated values), or **entropy regularisation** — explicitly adding the entropy of the policy to the objective:

$$J_{\text{reg}(\pi)} = J(\pi) + \beta \mathcal{H}(\pi)$$

where $\mathcal{H}(\pi) = -\mathbb{E}[\log \pi(a \mid s)]$ is the entropy. PPO uses entropy regularisation through the `ent_coef` hyperparameter, which we discuss in Section 3.8.2.

In financial RL, exploration carries an additional cost: **every exploratory action is a real trade with real transaction costs**. An agent that explores too aggressively can squander a substantial fraction of its training budget on random trades that never pay off. Our hyperparameter tuning consistently selects a **higher** entropy bonus (0.031 vs the default 0.01) than is typical in non-financial RL — reflecting the non-stationary nature of the problem and the need to keep exploring even after good policies emerge.

2.1.3 Value Functions and the Bellman Equations

Two key quantities in RL are the state-value function $V^{\pi(s)}$ and the action-value function $Q^{\pi(s,a)}$:

$$V^{\pi(s)} = \mathbb{E}_{\pi}[G_t \mid s_t = s] = \mathbb{E}_{\pi}[r_t + \gamma V^{\pi(s_{t+1})} \mid s_t = s]$$

$$Q^{\pi(s,a)} = \mathbb{E}_{\pi}[G_t \mid s_t = s, a_t = a]$$

The second equality for V^{π} is the Bellman expectation equation. The optimal policy π^* satisfies the Bellman optimality equations.

2.1.4 Q-Learning and Deep Q-Networks

Q-learning [8] estimates the optimal action-value function $Q^*(s, a)$ through iterative updates:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]$$

Deep Q-Networks (DQN) [9] approximate Q^* with a neural network and introduced two key stabilisation techniques: experience replay and target networks. DQN is limited to discrete action spaces and tends to overestimate action values.

2.1.5 Policy Gradient Methods

An alternative is to directly optimise the policy parameters θ by following the gradient of the expected return. The policy gradient theorem [10] states:

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t \right]$$

Actor-critic methods [11] reduce variance by replacing G_t with the advantage function $A^{\pi}(s_t, a_t) = Q^{\pi}(s_t, a_t) - V^{\pi}(s_t)$.

2.1.6 Proximal Policy Optimisation

Proximal Policy Optimisation (PPO) [12] is the algorithm used in this work. To understand PPO requires a brief detour through the **instability problem** of vanilla policy-gradient methods.

The instability problem. Consider an iteration of policy-gradient training: we collect a batch of trajectories under policy $\pi_{\theta_{\text{old}}}$, compute the gradient $\nabla_{\theta} J$, and take a step $\theta_{\text{new}} = \theta_{\text{old}} + \alpha \nabla_{\theta} J$. The issue is that **the gradient is only valid in a small neighbourhood of θ_{old}** . If α is too large, θ_{new} may correspond to a policy that visits radically different states than $\pi_{\theta_{\text{old}}}$, the trajectories already collected become unrepresentative of the new policy's behaviour, the next gradient estimate is even worse, and training diverges catastrophically. This phenomenon is known as **policy collapse** and is the dominant failure mode of naive policy-gradient methods.

Pre-PPO solutions. Trust Region Policy Optimisation (TRPO) [13] solved the instability problem by constraining each update to lie within a KL-divergence ball around the old policy:

$$\begin{aligned} & \max_{\theta} \hat{\mathbb{E}}_t \left[\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}} (a_t | s_t) \hat{A}_t \right] \\ & \text{subject to} \quad \hat{\mathbb{E}}_t [\text{KL}(\pi_{\theta_{\text{old}}}, \pi_{\theta})] \leq \delta \end{aligned}$$

TRPO requires **second-order** optimisation (conjugate gradient with Fisher information matrix), which is mathematically elegant but computationally expensive and finicky to implement.

PPO's contribution. PPO achieves **similar** stability with **first-order** gradient ascent by **clipping the probability ratio** directly in the objective. Define the probability ratio at step t as

$$r_{t(\theta)} = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}}(a_t | s_t)$$

This is 1 when the new policy makes the same choice as the old policy with the same probability, > 1 when the new policy is more likely to make this choice, and < 1 when less likely. PPO's clipped surrogate objective is:

$$L^{\text{CLIP}(\theta)} = \hat{\mathbb{E}}_t \left[\min \left(r_{t(\theta)} \hat{A}_t, \text{clip} \left(r_{t(\theta)}, 1 - \varepsilon, 1 + \varepsilon \right) \hat{A}_t \right) \right]$$

where $\hat{A}_t$ is the **advantage estimate** at step t — how much better action a_t was than the policy's average. The clipping operation has a specific intended behaviour:

- If $\hat{A}_t > 0$ (action was better than average), the unclipped term wants to **increase** r_t to make the action more likely. The clipping caps r_t at $1 + \varepsilon$, so once the new policy is ε more likely to take this action than the old policy, **no further increase is rewarded**. The objective becomes flat in θ beyond that point, bounding the update.
- If $\hat{A}_t < 0$ (action was worse), the unclipped term wants to **decrease** r_t . Clipping caps the decrease at $1 - \varepsilon$, similarly bounding the update.

The $\min$ outside the clip ensures the objective is **pessimistic**: when the clipped surrogate and the unclipped surrogate disagree, the smaller (more conservative) value is used. This prevents the agent from exploiting cases where the unclipped objective is **higher** than the clipped one (which would mean a large update that violates the trust region).

The total PPO loss combines the clipped policy surrogate with a value- function loss and an entropy bonus:

$$L^{\text{TOTAL}(\theta, \varphi)} = L^{\text{CLIP}(\theta)} - c_1 L^{\text{VF}(\varphi)} + c_2 \mathcal{H}(\pi_{\theta})$$

where $L^{\text{VF}(\varphi)} = \hat{\mathbb{E}}_t \left[\left(V_{\varphi}(s_t) - \hat{R}_t \right)^2 \right]$ is the value function loss with parameters φ , c_1 is the value-loss coefficient (typically 0.5), and c_2 is the entropy coefficient (ent_coef, in our tuned model 0.031).

Advantage estimation. The advantage $\hat{A}_t$ is computed using **Generalised Advantage Estimation** (GAE):

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

where $\lambda \in [0, 1]$ controls the bias-variance trade-off. $\lambda = 0$ gives the one-step TD error (low variance, high bias); $\lambda = 1$ gives the full Monte Carlo return (high variance, no bias). PPO with $\lambda \approx 0.95$ — the value we use — is a sensible default for most environments.

Why PPO for this problem. PPO is particularly well-suited for the present problem because:

1. **Discrete small action space.** The action space is $\{0, 1\}$ or $\{0, 1, 2\}$ — exactly the regime PPO was designed for. Algorithms like SAC are theoretically motivated for continuous action spaces and would require additional adaptation.
2. **Slow environment.** Each environment step is one simulated trading day; vectorising over four parallel environments yields 2×10^3 steps per wall-clock second. Sample efficiency (more learning per step) is less important than **stability** (no catastrophic divergence during 500,000 steps). PPO’s clipping bound provides exactly this stability.
3. **Entropy bonus aligns with exploration needs.** Financial environments are non-stationary; an agent that converges prematurely to a fixed policy is brittle. PPO’s `ent_coef` natively maintains exploration without requiring adversarial training-time tricks.
4. **Mature, well-tested implementations.** `stable-baselines3` provides a thoroughly tested PPO implementation [14]. We made a deliberate choice to use library PPO rather than re-implement, prioritising reduced bug risk over algorithmic novelty.

We did consider alternatives (DQN, A2C, SAC) and rejected them: DQN requires discrete actions but is prone to over-estimation, A2C is unstable under non-stationarity, SAC is overkill for two-action discrete problems.

2.1.7 The Soft Actor-Critic Alternative

Soft Actor-Critic (SAC) [15] extends the RL objective with a maximum entropy term and is theoretically well-motivated for continuous action spaces. For the discrete action space of this work, PPO is the more natural choice, but SAC is a natural extension for future work with continuous position sizing.

2.2 Reinforcement Learning for Algorithmic Trading

2.2.1 Historical Development

Early work by [16] applied Q-learning to stock trading with hand-crafted features. [17] extended this with deep neural networks on the KOSPI. FinRL [18] systematised the field by providing a standardised Gym environment [19], baselines, and data wrappers, and has been extended to multi-asset portfolio management [20] and risk-constrained formulations [21].

2.2.2 Key Challenges in Financial RL

Non-stationarity. Market dynamics change over time due to structural shifts, regime changes, and adaptation by other participants. We address this by evaluating on a held-out test split and training across multiple years and tickers.

Sparse and noisy rewards. Daily returns have an extremely low signal-to-noise ratio. We use a 5-day rolling risk-adjusted reward to average out some of the noise.

Transaction costs. Real trading incurs bid-ask spreads, commissions, and market impact. Our simulation applies a flat λ_{cost} on every position change.

Overfitting. RL agents readily exploit spurious historical patterns. We isolate test data and freeze design decisions before any test evaluation.

2.2.3 Reward Function Design

Raw returns are noisy and may incentivise excessive risk-taking. Risk-adjusted rewards such as the differential Sharpe ratio [22] can be computed incrementally:

$$R_t = \text{Sharpe}(r_{t+1:t+5}) = \sqrt{252} \cdot \frac{|(r)_{t+1:t+5}|}{\sigma_{r_{t+1:t+5}} + \varepsilon}$$

We use a Sortino variant of this design in the final v2 environment; the rationale and consequences are discussed in Sections 4.4 and 6.3.

2.3 Transformer-Based Language Models

2.3.1 The Transformer Architecture

The Transformer [23] is the neural architecture underlying all modern pre-trained language models. Before its introduction, sequence modelling was dominated by recurrent architectures (LSTMs [24], GRUs) that processed tokens left-to-right and maintained a hidden state. The recurrent design has two critical limitations: (i) it cannot be parallelised across the sequence dimension (each token must wait for the previous one), and (ii) information from distant tokens decays through the recurrent updates, making long-range dependencies hard to learn. The Transformer eliminates both limitations by replacing recurrence with **self-attention**.

Self-attention computes, for each token in a sequence, a weighted sum of all other tokens in the sequence. The weights are determined by the **compatibility** between a **query** vector Q (derived from the current token) and **key** vectors K (derived from every token, including itself):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

The mechanism can be read as follows: the dot product QK^T measures how much each token’s query “matches” each other token’s key. Scaling by $\sqrt{d_k}$ (where d_k is the key dimension) keeps the softmax in a non-saturated regime. The softmax converts the raw scores into a probability distribution over the sequence. Finally, this distribution is used to compute a weighted sum of **value** vectors V (also derived from each token), producing a context-dependent representation of the current token.

The key advantages over recurrent architectures are:

1. **Parallelism.** All token positions can be processed simultaneously during training because the computation depends only on Q , K , V which are fixed for the entire input. This is the technical pre-condition for training on billion-token corpora.
2. **Long-range dependencies.** Any two tokens can attend directly to each other regardless of their distance. There is no information loss through repeated recurrent updates.
3. **Interpretability.** The attention weights are inspectable. For each token, one can see exactly which other tokens contributed to its representation.

Multi-head attention extends single-head attention by running h attention heads in parallel with different learned projections W_Q^h , W_K^h , W_V^h , then concatenating their outputs. This allows different heads to attend to different aspects of the input — e.g., syntactic dependencies in one head, semantic relationships in another.

Positional encoding. Self-attention is **permutation-invariant**: a sequence (x_1, x_2, x_3) and (x_3, x_1, x_2) produce **the same set of attended representations**. To inject position information, a **positional encoding** vector is added to each token embedding. The original Transformer uses sinusoidal encodings; BERT uses **learned** positional embeddings (a separate embedding table indexed by position). The maximum sequence length supported by BERT-base is 512 tokens, set by the size of this learned embedding table.

The complete Transformer block stacks multi-head self-attention with a position-wise feedforward network, both wrapped in residual connections and layer normalisation:

$$\text{TransformerBlock}(x) = \text{LayerNorm}(x + \text{FeedForward}(\text{LayerNorm}(x + \text{MultiHeadAttention}(x))))$$

A Transformer encoder is a stack of L such blocks (12 in BERT-base, 24 in BERT-large), producing for each input token a contextual representation that depends on the entire input sequence.

2.3.2 BERT: Pre-training and Fine-tuning

BERT [1] is the first widely-deployed application of the **pre-training + fine-tuning** paradigm to natural language processing. It is a Transformer **encoder** (not a decoder, like GPT models) pre-trained on large text corpora with two self-supervised objectives:

Masked Language Modelling (MLM). In each training example, 15% of input tokens are randomly selected for masking. Of these, 80% are replaced with a special [MASK] token, 10% are replaced with a random token, and 10% are left unchanged. The model must predict the original token at each masked position from its bidirectional context. The masking scheme is carefully designed: the 10% random-replacement and 10% unchanged cases force the model to learn meaningful representations of **every** token rather than relying on the [MASK] signal as a shortcut.

Next Sentence Prediction (NSP). Pairs of sentences (A, B) are sampled from the corpus, with B being the actual successor of A 50% of the time and a random sentence 50% of the time. The model receives a special [CLS] token at the start of the input and must

predict from the [CLS] representation whether B follows A. NSP encourages BERT to learn document-level representations beyond single-sentence understanding. Subsequent work [25] showed that NSP can be removed without loss of downstream performance, but it was included in the original BERT and FinBERT inherits the resulting capability.

BERT-base configuration. 12 Transformer layers, 768-dimensional hidden states, 12 attention heads (each operating on 64-dim slices of the hidden state), and approximately 110 million parameters. The original BERT was pre-trained on BookCorpus (800M words) and English Wikipedia (2,500M words) for 1 million steps with batch size 256, requiring days on TPU clusters. The pre-training need only be done **once**; the resulting weights encode broad linguistic knowledge that can be cheaply adapted to downstream tasks.

Fine-tuning. Adapting BERT to a specific task requires adding a task- specific **head** (a small neural network, typically one or two layers) on top of the pre-trained encoder and training the **combined** model end-to-end on labelled task data. For classification, the head typically takes the [CLS] representation as input and produces logits over the task classes. Fine- tuning is dramatically cheaper than pre-training: it can be done on a single GPU in hours rather than days on TPU pods, and requires labelled datasets of thousands rather than billions of examples.

The pre-train + fine-tune recipe transformed NLP. The breakthrough insight is that **general linguistic competence** (syntax, morphology, semantic similarity, world knowledge) can be learned once from raw text and then **adapted** to almost any downstream task with a fraction of the data and compute. This is the same paradigm that underlies modern LLMs, scaled up by several orders of magnitude.

Why an encoder, not a decoder, for sentiment classification. Modern generative LLMs (GPT-4, Claude, Llama) are **decoder-only** Transformers trained for next-token prediction. They are powerful zero-shot classifiers via prompting (“Is this article bullish or bearish? Article: ...”), but they have several disadvantages for batch classification at our scale:

1. **Throughput.** Each article requires autoregressive generation of the output tokens, slower than a single encoder forward pass.
2. **Determinism.** LLM outputs are sensitive to prompt phrasing, temperature setting, and sampling configuration. Reproducible numeric scores require careful prompt engineering and temperature=0 sampling, which is itself **not exactly** deterministic across hardware.
3. **Cost.** At 157,804 articles and even at a low per-call price, processing the full corpus through an API-based generative language model would cost in the hundreds of dollars; a 7B local model would require GPU hours. FinBERT on a single Apple M-series chip processes the same corpus in 25 minutes at zero marginal cost beyond electricity.
4. **Calibration.** Encoder classifiers with a fixed softmax head produce well-calibrated probabilities suitable for the (score, confidence) mapping. “Logit-as-confidence” in

generative language models is well known to be poorly calibrated without explicit calibration steps.

The trade-off is **semantic depth**: a modern LLM can read a long earnings transcript, identify the conditional clauses, distinguish reassuring boilerplate from genuine guidance changes, and produce a nuanced reading that FinBERT cannot match. Section 6.4 and the future-work chapter revisit this trade-off.

2.3.3 FinBERT: Domain Adaptation for Financial Text

FinBERT [2] is a **domain-adapted** BERT-base model produced through a two-stage adaptation:

Stage 1 — Domain pre-training. The pre-trained BERT-base checkpoint was further pre-trained (using the same MLM + NSP objectives) on the **Reuters TRC2** news corpus, which contains approximately 1.8 billion words of financial newswire. This domain-adaptive pre-training updates the model’s internal representations so that financial vocabulary (EBITDA, guidance, forward P/E, covenant, accretive, restatement) is properly tokenised and contextualised. General BERT’s BookCorpus+Wikipedia training contains relatively few examples of financial-specific usage patterns.

Stage 2 — Task fine-tuning. The domain-adapted checkpoint was then fine-tuned on the **Financial PhraseBank** dataset [26] for three-class sentiment classification (positive, negative, neutral).

The result is a model that understands financial text qualitatively better than vanilla BERT and can be used out-of-the-box for sentiment scoring without further fine-tuning.

Why financial text needs its own model. Financial language has a distinctive vocabulary, writing style, and sentiment convention that differs substantially from generic English:

1. **Vocabulary.** Specialised terms (EBITDA margin, debt-to-equity, accretive acquisition, covenant breach, going concern, blackout window) are either out-of-vocabulary or under-represented in general corpora.
2. **Style.** Forward-looking statements are wrapped in safe-harbour disclaimers (“we expect”, “anticipate”, “subject to numerous risks”) that should not themselves be treated as predictions of future fact.
3. **Sentiment conventions.** “The company missed expectations by 2 cents” is **negative** even though “missed” carries a generic neutral connotation; “the company maintained guidance” is **neutral** even though “maintained” has a generic positive connotation. A general-purpose sentiment model trained on movie reviews or tweets gets these wrong systematically.

Why we do not further fine-tune FinBERT. We use FinBERT off-the-shelf without further fine-tuning, for three reasons:

1. **Lack of labelled in-domain data.** Fine-tuning requires labelled (text, sentiment) pairs in the specific style of our news corpus (Alpha Vantage news headlines + summaries). We do not have such labels at scale; the Alpha Vantage API provides its **own** pre-computed sentiment, but using that as a fine-tuning target would create a self-fulfilling loop in which we are predicting another model's predictions rather than ground truth.
2. **Risk of overfitting.** FinBERT has 110M parameters; the Financial PhraseBank training data on which it was fine-tuned has only 3,500 sentences. Adding a **second** fine-tuning stage on a small downstream dataset risks degrading the careful calibration achieved in the first stage.
3. **Cost of evaluation.** Each fine-tuning iteration would require re-running the full inference pipeline over 157,804 articles, re-aggregating to daily signals, and re-training the RL agent. The compute cost grows rapidly with the number of fine-tuning iterations.

The decision to use frozen FinBERT is therefore conservative: we accept whatever calibration biases the off-the-shelf model has (including the neutral→positive bias documented in Section 3.5.5) in exchange for not introducing new biases from a small-data fine-tuning step.

Score and confidence extraction. FinBERT produces a 3-class softmax (p_+, p_-, p_0) with $p_+ + p_- + p_0 = 1$. From these we derive the article-level (score, confidence) pair:

$$s_i = p_+^i - p_-^i \in [-1, 1], \quad c_i = \max(p_+^i, p_-^i) \in [0, 1]$$

The score s_i measures the net direction of sentiment — positive for bullish, negative for bearish, near zero when the model is uncertain or predicts neutral. The confidence c_i measures how strongly the model commits to either non-neutral direction, **deliberately excluding the neutral probability mass**. A score of +0.8 with confidence 0.9 is fundamentally different from a score of +0.8 with confidence 0.5:

- $(s = +0.8, c = 0.9)$ implies $(p_+, p_-, p_0) \approx (0.85, 0.05, 0.10)$ — the model is strongly bullish with little hedging.
- $(s = +0.8, c = 0.5)$ implies $(p_+, p_-, p_0) \approx (0.50, -0.30, 0.80)$ — but this is **impossible** because $p_- \geq 0$. A valid decomposition consistent with $s = +0.8$ and lower confidence is $(0.85, 0.05, 0.10)$ exactly — the example shows that c is functionally redundant with s at high scores.

The independence of s and c is most informative **at intermediate scores**: a score $s = +0.2$ could correspond to $(p_+, p_-, p_0) = (0.4, 0.2, 0.4)$ ($c = 0.4$, hedged) or $(0.55, 0.35, 0.10)$ ($c = 0.55$, confidently mildly bullish). Our (score, confidence) representation preserves both pieces of information, which a one-dimensional summary (just s , or just argmax class) would conflate.

2.4 LLMs for Financial Sentiment and Forecasting

Financial sentiment analysis was originally dictionary-based — the Loughran–McDonald list [3] being the most influential — before moving to supervised ML and pre-trained encoders. [27] shows that the fraction of negative words in WSJ columns predicts next-day returns and volume. [28] demonstrates that opinion-based forum sentiment is correlated with stock prices.

[29] evaluate ChatGPT’s zero-shot ability to predict stock price movements from news headlines and find statistically significant return predictability, particularly for stocks with high retail attention and during periods of high news uncertainty. BloombergGPT [30] is a 50B-parameter model pre-trained exclusively on financial text. Such large models are computationally prohibitive for inference at the scale required for backtesting (157,804 articles in our dataset), which motivates FinBERT as a practical baseline.

2.5 Integration of LLMs and Reinforcement Learning for Trading

2.5.1 Early-Fusion Approaches

[6] train a PPO agent on a state vector concatenating price features and LLM-derived sentiment embeddings, evaluated on AAPL and the LEXCX mutual fund. They report improvements in Sharpe and cumulative profit over a price-only baseline. Their work is the closest predecessor to this thesis, differing in that they do not implement the (score, confidence) abstraction or study dynamic routing.

FLAG-Trader [31] fine-tunes an LLM end-to-end as the **policy network itself**, using LoRA to avoid updating all parameters. Trading-R1 [32] introduces chain-of-thought reasoning into the trading loop using PPO over a corpus of 100,000 financial training samples. ElliottAgents [33] proposes a multi-agent system combining classical chartist analysis with retrieval-augmented LLM news processing [34]; the combination is rule-based rather than learned.

2.5.2 Gap in the Literature

A systematic review of 84 papers published between 2022 and 2025 [35] confirms that all existing LLM+RL trading systems adopt some form of early fusion; none implement an explicit (score, confidence) abstraction layer or study quantitatively how the relative contribution of each signal source varies across market regimes. Three specific gaps motivate the present work: (i) no system enables independent module evaluation; (ii) no system provides quantitative routing evidence; (iii) confidence — how certain each module is — has not been modelled as an explicit input to the fusion layer.

3 System Design and Implementation

3.1 Overall Architecture

The system has three components in cascade:

1. **Sentiment module:** FinBERT $\rightarrow$ daily (score, confidence) per asset.
2. **Technical module:** 15 price/volume features $\rightarrow$ XGBoost $\rightarrow$ daily (score, confidence) per asset.
3. **Fusion model:** PPO over the 4 module outputs plus 3 lagged returns $\rightarrow$ trading action.

The architectural invariant is **strict modularity**: each component can be trained, evaluated, and replaced independently. The fusion model sees **homogeneous** inputs in $[-1, 1] \times [0, 1]$ regardless of whether the signal originated as text or price. Figure 3.1 summarises the data flow.

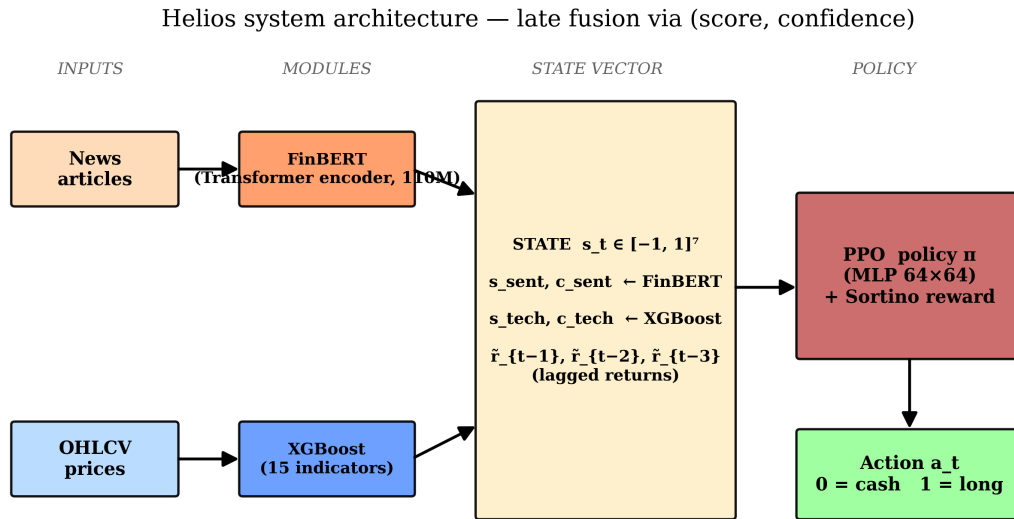


Figure 1: System architecture. Two independent modules — FinBERT for text, XGBoost for prices — emit a standardised (score, confidence) pair into a PPO fusion policy that also sees three lagged returns. The fusion model is the only component trained end-to-end with the trading reward.

3.2 Data Collection

3.2.1 Equity Price Data

Daily OHLCV (Open, High, Low, Close, Volume) was downloaded for 10 large-cap S&P 500 constituents. The complete list (with sectors and row counts) is given in Table 3.1.

Ticker	Company	Sector	Rows
AAPL	Apple Inc.	Technology	2,010
MSFT	Microsoft Corp.	Technology	2,010
NVDA	NVIDIA Corp.	Semiconductors	2,010
AMZN	Amazon.com Inc.	Consumer/Cloud	2,010
GOOGL	Alphabet Inc.	Technology	2,010
META	Meta Platforms Inc.	Social Media	2,010
JPM	JPMorgan Chase	Financials	2,010
LLY	Eli Lilly and Co.	Healthcare	2,010
AVGO	Broadcom Inc.	Semiconductors	2,010
TSLA	Tesla Inc.	Automotive/EV	2,010

Table 1: Ten large-cap S&P 500 constituents in the universe.

The data covers 2 January 2018 – 30 December 2025 (2,010 trading days per ticker, no missing values), obtained via the `yfinance` library [36] with `auto_adjust=True` so splits and dividends are back-adjusted.

3.2.2 Financial News Data and Temporal Coverage

Financial news was obtained from the Alpha Vantage News Sentiment API. Each API query is made **per ticker symbol**: the endpoint returns articles that the provider’s own NLP pipeline has associated with that specific company. We therefore rely on Alpha Vantage’s ticker-level tagging for the article-to-company assignment, which covers direct mentions and company aliases but not complex multi-entity references.

For each article we retain only the title, summary, and UTC publication timestamp; the API’s pre-computed sentiment is used only as a sanity check, not as a model input. Trading days for which no articles are returned produce a zero-confidence sentinel ($s_{\text{sent}} = 0, c_{\text{sent}} = 0$), which the RL agent learns to treat as “no information”; this accounts for approximately 37% of training days and 10% of test days (where news coverage is denser). The full corpus contains 157,804 articles, distributed across tickers as shown in Table 3.2.

Ticker	Articles	Earliest	Latest
AAPL	12,325	2018-01-01	2026-04-28
MSFT	23,286	2018-01-04	2026-04-28
NVDA	26,550	2018-01-04	2026-04-28
AMZN	19,052	2018-01-01	2026-04-28
GOOGL	13,983	2018-01-03	2026-04-28
META	6,175	2018-01-01	2026-04-28
JPM	33,673	2018-01-01	2026-04-28
LLY	5,674	2018-01-07	2026-04-28
AVGO	7,199	2018-01-18	2026-04-28
TSLA	9,887	2018-01-05	2026-04-28

Table 2: News-article counts per ticker (Alpha Vantage download).

Temporal coverage of news vs. prices. The news download was last refreshed in April 2026, so the raw corpus contains articles published up to 2026-04-28. However, the **price** time series ends on 2025-12-30. Any article whose alignment date falls beyond 2025-12-30 (the last test trading day) is **dropped** before splitting: it cannot be assigned to any valid trading day in the working universe and is therefore not used by FinBERT, by the sentiment aggregation, by the technical module, or by the RL training. The 2026-Q1 articles exist only in the raw download archive as a by-product of when the data extraction was last run; they have no causal influence on any result reported in this thesis.

Coverage skew. Article density is heavily skewed toward recent years. For AAPL, 2,700 articles cover all of 2018–2022 (5 years), while 2025 alone contributes over 4,100 articles. This reflects both the growth of financial media and the depth of Alpha Vantage’s historical indexing, and has direct implications for the sentiment module’s behaviour across splits, discussed in Section 3.5 and listed as a limitation in Chapter 8.

3.3 Temporal Alignment and Leakage Prevention

Data leakage in financial ML refers to using information that would **not** have been available to a real-time trader at the moment of decision. It is one of the most insidious sources of overly optimistic backtests and receives explicit systematic treatment here [37].

3.3.1 The Leakage Problem

Consider a news article published at 14:30 ET on a given trading day. The NYSE closes at 16:00 ET. Using that article to predict the same day’s close means the model has seen information published mid-session — a real-time trader making a decision at market open (09:30 ET) could not have read it. More subtly, using the same-day close as a target with any same-day feature creates look-ahead bias.

3.3.2 Alignment Rules

Rule	Operational definition
R1 — Market-open cutoff	An article timestamped at t_{pub} (UTC) is assigned to the next NYSE session whose 09:30 ET market open is strictly after t_{pub} . Anything published after 09:30 ET on day T is shifted to day $T + 1$.
R2 — Weekend / holiday propagation	Articles published on non-trading days (weekends, NYSE holidays) propagate to the next valid trading session, using <code>pandas_market_calendars</code> .
R3 — Filing embargo	SEC 10-K / 10-Q filings receive an additional 24-hour embargo beyond R1 to account for after-hours publication and digestion time. A filing on day T becomes a feature from day $T + 2$ onwards.
R4 — Train-only normalisation	All price-derived features are z-scored using μ and σ computed only on the training split. The same parameters are then applied to validation and test without recomputation.

Table 3: Temporal alignment rules applied throughout the pipeline.

3.3.3 Worked Example

Consider an AAPL article published on **Friday 15 March 2024 at 18:47 UTC** (14:47 ET).

- **R1:** 14:47 ET on day T is after 09:30 ET, so the article cannot be used as a feature for day T . Earliest eligible session is Monday 18 March 2024.
- **R2:** 18 March 2024 was an open NYSE session (no holiday), so it remains the target session.
- **R3:** This is a news article, not an SEC filing — no additional embargo applies.
- **Final:** the article enters the daily aggregate for AAPL on 2024-03-18.

A second case: an AAPL 10-Q filing timestamped **Thursday 1 February 2024 at 21:35 UTC** (16:35 ET, after close).

- **R1:** shifted to Friday 2 February 2024.
- **R3:** extra 24-hour embargo → earliest eligible session is Monday 5 February 2024.

3.3.4 Automated Verification

An `assert_no_leakage` function checks that for every (article, feature-date) pair the publication timestamp is strictly before market open of the assigned date. It ran clean on all three splits — **zero violations across 90,601 news-day pairs** — and is included in the project’s automated test suite.

3.4 Dataset Splits

Split	Period	Rows	Avg news/day	Label=1 %
Train	2018-01-01 – 2022-12-31	12,590	1.79	56.0%
Val	2023-01-01 – 2023-12-31	2,500	2.01	61.1%
Test	2024-01-01 – 2025-12-31	4,960	12.73	58.2%

Table 4: Train / validation / test splits and label balance.

Training (2018–2022) spans late-cycle bull market, COVID-19 crash and recovery, speculative bull 2020–21, and bear market 2022. **Validation (2023)** is the AI-driven mega-cap rally and is used only for threshold and hyperparameter selection. **Test (2024–2025)** spans 496 trading days under macroeconomic uncertainty — tariff escalations, AI capex debate, sector rotation.

The label $y_t = \mathbb{1}[r_t^{5d} > 0]$ is balanced at 56–61% positive across splits, requiring no resampling.

3.5 Sentiment Analysis Module

3.5.1 Architecture and Model Selection

The sentiment module is built on FinBERT (BERT-base, 110M parameters, 12 layers, 768 hidden dims). Inference over the full 157,804-article corpus takes 25 minutes on Apple M-series MPS at 2 batches of 32 articles per second. A 7B-parameter generative LLM would require GPU-hours or significant API spend; a 70B model is computationally prohibitive at this scale. FinBERT’s F1 of 0.876 on Financial PhraseBank shows it extracts real signal, making the efficiency–accuracy trade-off favourable.

3.5.2 Input and Score Computation

For each article we concatenate title and summary with [CLS]/[SEP] tokens and feed up to 500 sub-tokens. FinBERT outputs (p_+, p_-, p_0) with $p_+ + p_- + p_0 = 1$. We map this to:

$$s_i = p_+^i - p_-^i \in [-1, 1], \quad c_i = \max(p_+^i, p_-^i) \in [0, 1]$$

A score of +0.8 with confidence 0.9 is very different from +0.8 with confidence 0.5: in the latter case the model assigns significant mass to **negative**, indicating genuine ambiguity rather than weak confidence in the positive class.

3.5.3 Daily Aggregation

The aggregation step compresses N article-level (score, confidence) pairs on a given (asset, day) into a single $(s_{\text{sent}}^t, c_{\text{sent}}^t)$ pair. The specific aggregation formula matters; we consider three alternatives, justify the choice we made, and discuss its implications.

Alternative A — Simple mean. The simplest option is to average article scores and confidences independently:

$$s_t^A = \frac{1}{N} \sum_i s_i, \quad c_t^A = \frac{1}{N} \sum_i c_i$$

This treats every article as equally informative, which is **wrong**: an article that FinBERT classifies as neutral (low c_i) contributes the same to the daily score as one classified confidently positive (high c_i). The effect is that on busy news days, a few highly confident bullish articles can be “averaged away” by many neutral ones, producing a near-zero daily score that does not reflect the underlying signal.

Alternative B — Confidence-weighted mean (chosen). We instead use a **confidence-weighted** mean for the score and an unweighted mean for the confidence:

$$s_{\text{sent}}^t = \frac{\sum_i c_i \cdot s_i}{\sum_i c_i}, \quad c_{\text{sent}}^t = \frac{1}{N} \sum_i c_i$$

Each article’s contribution to the daily score is **weighted by its own confidence**: a confidently bullish article contributes more than a borderline one. The daily confidence is the **average** confidence across all articles — a day with 10 high-confidence articles is more informative than a day with 1 high-confidence article, but the per-article confidence is what matters most.

Alternative C — Hierarchical Bayesian. A more sophisticated approach would treat each article as a noisy observation of a latent daily sentiment and infer the posterior over the daily sentiment using Bayes’ rule with informative priors. This would correctly handle uncertainty propagation and outlier-robust aggregation. We rejected this for three reasons: (i) implementation complexity not warranted at this scale; (ii) the resulting daily distribution is hard to compress into a (score, confidence) pair without losing information; (iii) the gain is theoretical without strong empirical motivation.

The empty-day case. When no articles are aligned to a given trading day (approximately 37% of training days, 10% of test days), the aggregation formula is undefined ($\sum_i c_i = 0$). The implementation defaults to $(s_{\text{sent}}^t, c_{\text{sent}}^t) = (0.0, 0.0)$ — explicitly signalling to the fusion model “no sentiment information available”. This is **not** the same as “neutral sentiment with high confidence”, which would be encoded as $(0.0, 1.0)$. The fusion model learns to treat these two cases differently: zero confidence means “ignore the sentiment channel”; high confidence at score zero means “the news exists and is genuinely balanced”.

The empirical effect of the empty-day default is visible in the regime analysis (Section 5.7): on `low_news` days the agent’s `conf_sent` is exactly zero and the agent’s behaviour shifts toward technical-and-momentum-based decisions. This is the **most direct** observable manifestation of dynamic routing in the system.

Why a single daily aggregate rather than article-level state. An alternative design would feed the RL agent the **full sequence** of article (score, confidence) pairs for the day, perhaps via a small Transformer or RNN inside the policy network. We rejected this for

two reasons: (i) it would require much more training data to fit the inner sequence model; (ii) it would re-introduce the early-fusion problem of opaque feature interactions inside the policy. The daily aggregate is a deliberate **information bottleneck** that forces the upstream pipeline to commit to a summary the agent can interpret.

3.5.4 Standalone Evaluation on Financial PhraseBank

Metric	Value
Accuracy	0.876
F1 macro	0.876
Negative-class recall	0.987
Positive-class recall	0.974
Neutral-class recall	0.812
Neutral $\rightarrow$ Positive misclassifications	218 / 1,391

Table 5: FinBERT performance on Financial PhraseBank sentences_allagree (2,264 sentences).

The model achieves near-perfect recall on positive (97.4%) and negative (98.7%) classes. The dominant failure mode is **neutral** $\rightarrow$ **positive** (15.7%): financial news uses carefully optimistic language even in neutral contexts (“the company maintained guidance”), and FinBERT has learned that association. This produces a **systematic positive bias** in the aggregate scores (mean +0.16 to +0.31 across splits), which the fusion model must learn to discount during training.

3.5.5 Predictive Signal Validation

On the training split, restricting to days with at least one article ($n = 7,963$), the Pearson correlation between s_{sent}^t and the 5-day binary label y_t is $r = +0.036$ ($p = 0.0014$). Small but statistically significant. The same signal is visible in a quintile decomposition:

Quintile	Score range	Mean label_5d
Q1 (most bearish)	$s < -0.26$	0.519
Q2	$-0.26 \leq s < 0.06$	0.543
Q3	$0.06 \leq s < 0.32$	0.539
Q4	$0.32 \leq s < 0.60$	0.562
Q5 (most bullish)	$s \geq 0.60$	0.573

Table 6: Mean 5-day positive-return rate per sentiment quintile (training split, news days only).

A monotonic 5.4-percentage-point spread from Q1 to Q5 confirms that the FinBERT-based score carries genuine, if modest, predictive information.

3.5.6 Score Distribution Across Splits

Split	Days w/ news	Mean	Std	Mean conf.	conf > 0.2
Train	7,963 / 12,590 (63%)	+0.159	0.450	0.383	57.9%
Val	1,896 / 2,500 (76%)	+0.157	0.505	0.461	68.4%
Test	4,464 / 4,960 (90%)	+0.306	0.452	0.587	86.5%

Table 7: Daily aggregate sentiment statistics across splits.

Confidence-rich days range from 58% (train) to 87% (test) — qualitatively different from the technical module (0% across all splits). Both mean score and mean confidence increase from train to test, reflecting the Alpha Vantage coverage skew. This asymmetry is acknowledged as a limitation in Chapter 8.

3.6 Technical Analysis Module

3.6.1 Feature Engineering

The module computes 15 features per ticker-day from OHLCV data, grouped into momentum (`rsi_14`, `macd`, `macd_signal`, `macd_hist`), trend (`ema_20`, `ema_50`, `ema_cross`), volatility (`bb_upper`, `bb_mid`, `bb_lower`, `bb_pct`), return (`ret_5`, `ret_10`, `ret_20`) and volume (`volume_ratio`). All rolling windows are within-ticker only, and the first 20 warm-up rows per ticker are dropped.

3.6.2 XGBoost Classifier

XGBoost [38] was chosen for: (i) well-calibrated probabilities — essential to the (score, confidence) mapping; (ii) interpretable feature importances; (iii) very fast training at this dataset size, enabling extensive hyperparameter search.

3.6.3 Hyperparameter Tuning

50 Optuna trials [39] searched the following grid with early stopping on val AUC (patience 20):

Hyperparameter	Search range
<code>max_depth</code>	3–7 (integer)
<code>n_estimators</code>	100–500 (integer)
<code>learning_rate</code>	0.01–0.2 (log-uniform)
<code>subsample</code>	0.6–1.0 (uniform)
<code>colsample_bytree</code>	0.6–1.0 (uniform)
<code>min_child_weight</code>	1–10 (integer)
<code>gamma</code>	0.0–1.0 (uniform)

Table 8: XGBoost Optuna search space.

3.6.4 Why the Technical Score Is So Weak

The best Optuna trial reaches val AUC = **0.5188**. Early stopping selects effectively **one tree**. This is **not** a bug — it is a non-trivial finding about the dataset. Several plausible contributors:

1. **Feature engineering.** The 15 features are standard textbook indicators with no asset-specific calibration. Higher-frequency micro-structure data (intraday order-book imbalance, dark-pool prints), cross-sectional features (industry momentum) or longer-horizon factors (12-1 momentum, value signals) are not included. The chosen feature set is intentionally narrow to keep the technical module a **transparent baseline** rather than a pre-trained competitor.
2. **Prediction horizon.** The 5-day binary label averages over an interval that is short enough to be dominated by idiosyncratic noise yet long enough that the short-horizon momentum signals (3-day, 5-day reversals) studied by [40] are partially diluted.
3. **Probability calibration.** XGBoost is normally well-calibrated for binary classification, but the very small effective tree count produced by early stopping means $P(\text{up}|\mathbf{x})$ stays close to the training-set base rate for almost every input. The collapse is **information-theoretic** (no exploitable signal), not a calibration problem we can fix with isotonic regression.
4. **Market difficulty.** On a universe of 10 mega-cap, heavily-followed S&P 500 names, the **weak-form EMH** should hold quite tightly: any persistent textbook-indicator pattern would be arbitrated out by the systematic players that dominate volume in these names.

In practice the technical module’s probabilities almost never leave $[0.489, 0.511]$, giving scores in $[-0.022, +0.022]$. We **do not** conclude from this that technical analysis is universally useless — only that, on **this** universe, **this** prediction horizon, and **this** feature set, an honest baseline gradient-boosted classifier extracts essentially no signal. The fusion model retains the technical channel anyway, because (i) it is a zero-cost addition to the state, (ii) it functions as a sentinel of “near-zero confidence everywhere” that the agent learns to discount, and (iii) it is required by the late-fusion abstraction.

3.6.5 Score Construction

$$s_{\text{tech}}^t = 2P(\text{up} \mid \mathbf{x}_t) - 1, \quad c_{\text{tech}}^t = |s_{\text{tech}}^t|$$

Top-5 features by gain: `bb_mid`, `ema_20`, `ema_50`, `bb_lower`, `bb_upper` — all level/trend indicators, consistent with the weak-form momentum literature [40].

3.7 Trading Environment

3.7.1 Gym Interface

The environment is a `gymnasium.Env` subclass with the standard `reset()` and `step(action)` API. The `split(train, val, test)` is a constructor parameter.

3.7.2 State Space

The observation is a 7-dimensional continuous vector:

$$\mathbf{s}_t = [s_{\text{sent}}^t, c_{\text{sent}}^t, s_{\text{tech}}^t, c_{\text{tech}}^t, \tilde{r}_{t-1}, \tilde{r}_{t-2}, \tilde{r}_{t-3}]$$

with $\tilde{r}_{t-k} = \frac{\text{clip}(r_{t-k}, -0.1, 0.1)}{0.1} \in [-1, 1]$. The observation space is the hypercube $[-1, 1]^7$, ensuring all inputs are bounded and on a comparable scale.

The state design embodies four explicit choices, each with rationale:

Choice 1 — Why exactly these 7 dimensions, not more or fewer. The state must satisfy two competing constraints. It should be **rich enough** to support a useful policy (a state of one dimension would be informationally inadequate) and **narrow enough** not to overfit on the training dataset (12,590 ticker-days, small by RL standards). The (score, confidence) outputs of the two upstream modules already represent compressed summaries of the entire sentiment and price-history information; adding the four scalars they produce is therefore high-leverage. The three lagged returns provide the agent with **direct evidence of short-term momentum** without requiring it to reconstruct momentum implicitly through the technical score. We tested augmenting the state with 5 and 10-day lagged returns; this produced marginal improvement at the cost of more overfitting in cross-validation, so the final design uses 3 lags.

Choice 2 — Why three lagged returns specifically. Returns at lags 1, 2, 3 capture the immediate trend (positive/negative/positive sequences vs monotone sequences) without redundancy with the technical module’s longer-horizon indicators (ret_5, ret_10, ret_20). Three lags is enough to encode the sign-pattern of the last few days; more would correlate strongly with the technical features already encoded in s_{tech} , providing little marginal information.

Choice 3 — Why normalised, clipped returns. Raw daily returns have a long-tailed distribution: a typical day is $\approx \pm 1\%$ but the tails extend to $\pm 20\%$ during stress events. Feeding raw returns to a neural network creates two problems: (i) the network’s input layer is dominated by rare large values, distorting normalisation statistics, and (ii) gradient updates during stress days become disproportionately large, destabilising training. Clipping at $\pm 10\%$ and rescaling to $[-1, +1]$ solves both issues at the cost of losing magnitude information for $|r| > 10\%$ events — a trade-off justified by the rarity of such events and the architectural commitment to bounded inputs.

Choice 4 — Why $[-1, +1]^7$ rather than $\mathbb{R}^7$. A bounded observation space has three practical advantages: (i) initialisation of the policy network’s first layer is well-defined (Xavier or He init both assume bounded inputs); (ii) the gradient norm of the policy update is bounded above, contributing to PPO’s stability; (iii) Optuna hyperparameter search converges faster when the search space is dimensionally homogeneous. The cost is that score and confidence values cannot encode “absolute extreme” signals; this is acceptable because the upstream modules are themselves designed to produce bounded outputs.

3.7.3 Action Space (v1 vs v2)

Two versions of the action space coexist in the codebase, corresponding to the diagnostic iteration described in Section 4.4:

- **v1:** $\mathcal{A} = \{0, 1, 2\} \rightarrow \{\text{sell} = -1, \text{hold} = 0, \text{buy} = +1\}$
- **v2:** $\mathcal{A} = \{0, 1\} \rightarrow \{\text{hold/cash} = 0, \text{buy/long} = +1\}$ — **long-only**

Portfolio value evolves as $V_t = V_{t-1}(1 + p_{t-1}r_t)$. A flat cost λ_{cost} is debited on every position change.

3.7.4 Transaction Cost

The transaction cost coefficient is $\lambda_{\text{cost}} = 0.001$ in all reported experiments (i.e. **10 basis points** per position change). This is applied uniformly to every position flip, regardless of which direction. A flip from $p_{t-1} = 0$ to $p_t = +1$ or back costs 0.001 on the portfolio weight. The choice approximates retail-broker effective costs for liquid US mega-caps (commission-free brokers with typical bid-ask spread, modest market impact for small notional). A sensitivity study at $\{0, 5, 10, 20, 50, 100\}$ bps is reported in Section 5.8.

3.7.5 Episode Structure

Each **episode** corresponds to one **ticker-year**: one randomly selected ticker from the training pool, evaluated over the trading days of a single calendar year (typically 251–253 steps). At the end of the year, the environment signals `terminated=True` and the next `reset()` samples a fresh ticker-year pair.

Why episodic at the year boundary. The choice between (i) one long multi-year episode per ticker, (ii) one episode per year per ticker, (iii) shorter episodes (months, quarters), or (iv) continuous training without episode boundaries is non-trivial. We chose option (ii) for the following reasons:

1. **Stable advantage estimation.** PPO computes advantages over each rollout using GAE; rollouts that span market regime changes within a single episode produce high-variance advantage estimates. A one-year episode contains roughly one full annual cycle (earnings seasons, year-end positioning) and is short enough that regime mixing within the episode is moderate.
2. **Easy diversification.** Sampling ticker-year pairs uniformly during training guarantees the agent sees a mix of bull years (2020–21, 2023), bear years (2018Q4, 2022), and volatile sideways years (2019), independent of any single ticker’s idiosyncratic story. An agent trained only on NVDA 2020–21 would learn aggressive long-trend-following; an agent trained on a uniform sample of (ticker, year) pairs learns more robust policies.
3. **Computational tractability.** Each year-episode is 250 steps; collecting a rollout of `n_steps=2048` requires 8 episodes per environment. With four parallel environments this provides good rollout diversity per update.

The multi-episode training regime as implicit regularisation. An agent trained on a **single** long trajectory (one ticker, multiple years) would learn a policy specific to that ticker’s microstructure. The multi-episode setup forces the agent to discover policies that work **on average** across all ticker-year pairs in training — implicit regularisation against ticker-specific overfitting. The per-asset analysis in Section 5.6 confirms the result: the late-fusion agent generates positive cumulative return on **all 10 test tickers** without any failures, even though it was trained without ticker identity as an input feature.

Why ticker identity is not in the state. A natural addition to the state would be a one-hot ticker embedding. We deliberately omit it: including ticker identity would allow the agent to learn ticker-specific policies that might not generalise to new tickers. The omission forces the agent to rely on **signal-quality features** (sentiment confidence, technical confidence, return momentum) that are universal across assets. The cost is that the agent cannot exploit ticker-specific patterns (e.g., AAPL earnings calendar); the benefit is that the resulting policy generalises to any new liquid mega-cap stock without retraining.

3.7.6 Reward Function

The reward function is, after the state and action spaces, the most consequential design choice in the system. It encodes the agent’s objective and shapes everything it learns. We discuss the v1 (Sharpe-based) and v2 (Sortino-based) variants in detail in Section 4.4; here we describe the common structure.

Common form.

$$R_t = \text{RiskAdj}(r_{t+1:t+5}) - \lambda_{\text{cost}} \cdot \mathbb{1}[a_t \neq a_{t-1}]$$

The reward at step t has two components: a **forward-looking risk-adjusted return** term over the next 5 days, and a **transaction cost penalty** applied whenever the action changes from the previous step.

Why a 5-day forward window. A single-day reward r_{t+1} is extremely noisy — single-day returns have a signal-to-noise ratio so low that gradient estimates are essentially random. Averaging over a forward window reduces noise but introduces lag: longer windows give cleaner signal but with longer credit-assignment delay. We chose 5 days as a compromise between three considerations:

1. **Matches the prediction horizon of the technical module.** The XGBoost classifier predicts the sign of the 5-day forward return; using the same horizon in the reward makes the technical signal directly relevant.
2. **Matches typical “swing trading” horizon.** Five trading days corresponds to about a week — short enough that intra-week mean-reversion and news catalysts dominate, long enough that single-day noise averages out.
3. **Manageable computational requirement.** A 5-day forward window means each step’s reward requires only 5 prices into the future, easily looked up in the price array. Longer windows would require careful handling of episode boundaries.

Why clipping the risk-adjusted ratio. Both Sharpe and Sortino formulas contain a σ in the denominator. When σ is small (a calm, slowly- trending 5-day window) and the mean is non-zero, the raw ratio can be arbitrarily large in magnitude. PPO computes gradients proportional to the reward; large rewards produce large gradient updates that destabilise training. We clip the ratio to $[-3, +3]$ to bound this effect. The clipping threshold is empirically chosen: at $|R| = 3$ the agent’s annualised Sharpe in a single 5-day window would have to be $> 3\sqrt{252} \approx 47$, which is statistically nonsensical and almost certainly the result of a near-zero denominator. Clipping at 3 retains the genuine signal while removing artefactual extremes.

Why the $\sigma = 0$ degeneracy default. When σ is **exactly** zero (can occur if all 5 returns are equal, or near year-end with reduced trading days), the risk-adjusted ratio is undefined ($\frac{0}{0}$ in the limit). We replace it with $+0.5$ if the mean is positive and -0.5 if negative — a **deterministic** default that preserves the **sign** information without producing infinity. The magnitude 0.5 is chosen to be comparable in scale to typical risk-adjusted values during training, avoiding bias toward under- or over-rewarding these rare cases.

Why a flat per-flip transaction cost. The transaction cost is applied **once** per position change, regardless of the size or direction of the change. Real-world costs scale roughly linearly with the **change** in notional position, which would translate to $\lambda_{\text{cost}} \cdot |p_t - p_{t-1}|$. In our binary-position setup, $|p_t - p_{t-1}| \in \{0, 1\}$, so the linear and indicator forms coincide. If the action space were continuous, a linear penalty would be more appropriate.

Forward vs lagged reward — the temporal indexing question. At step t the agent picks action a_t , which determines position p_t for the **next** day. The reward at step t uses returns $r_{t+1:t+5}$. This means the reward correctly **credits the action with the outcome it caused**: action at t causes position p_t , which experiences return r_{t+1} overnight. The common alternative — rewarding a_t with r_t — is a **causal violation** that allows the agent to “see” the day’s return before choosing its action, a form of look-ahead bias that produces artificially good backtest results. Our temporal indexing is verified by the environment unit tests.

3.7.7 Environment Validation

Six unit tests pass: (1) 100 random episodes run without error; (2) all rewards finite; (3) all observations in $[-1, 1]$; (4) hold incurs no cost; (5) buy→sell incurs exactly one λ_{cost} ; (6) episode length matches expected trading days.

3.8 Fusion Model

3.8.1 Architecture

A `MlpPolicy` from `stable-baselines3` [14]: two hidden layers of 64 units each with ReLU, separate actor and critic networks. The small size is deliberate — the state space is only 7-dim and the dataset is 12,590 ticker-days; a deeper or wider net would overfit.

3.8.2 Base Training Configuration

Parameter	Value	Role
learning_rate	3×10^{-4}	Adam step size
n_steps	2,048	Steps per env before update
batch_size	64	Mini-batch size
n_epochs	10	Update passes per rollout
ent_coef	0.01	Entropy bonus
gamma	0.99	Discount factor
clip_range	0.2	PPO ϵ
total_timesteps	500,000	Total env steps

Table 9: Base PPO training configuration.

Four parallel environments via DummyVecEnv accelerate experience collection.

3.8.3 Hyperparameter Tuning

20 Optuna trials over learning_rate, ent_coef, gamma and lambda_cost, each trained for 200,000 timesteps and evaluated on 10 val episodes. Best trial (val Sharpe +1.954):

Parameter	Base	Tuned
learning_rate	3×10^{-4}	2.38×10^{-4}
ent_coef	0.01	0.031
gamma	0.99	0.9587
lambda_cost	0.001	0.001067

Table 10: Final PPO hyperparameters.

The lower learning rate stabilises updates; the higher entropy bonus encourages exploration appropriate for a non-stationary financial environment; the slightly lower γ reflects the short 5-day reward horizon.

4 Methodology and Experimental Design

4.1 Why an Ablation Study

The central methodological choice of this thesis is to evaluate the late- fusion hypothesis through a controlled **ablation study** rather than through (i) a tournament against external benchmarks, (ii) a leaderboard submission, or (iii) a published-method reproduction. The reasons are worth stating explicitly because they shape what conclusions can and cannot be drawn from the results.

Ablation isolates the architectural variable. A tournament against published methods would confound multiple variables at once: different universes, different time periods, different cost models, different hyperparameter budgets. An ablation holds **every** variable constant except the **one** of interest — the architectural choice between early and late fusion — making the resulting comparison genuinely causal.

Ablation respects scale constraints. A Master’s-thesis project cannot plausibly compete on raw Sharpe with a hedge fund’s production system that benefits from years of refinement and far richer data. An ablation deliberately reframes the question from “can this beat the market?” to “given identical resources, does architectural choice X dominate architectural choice Y?” — a question that can be honestly answered at this scale.

Ablation produces falsifiable hypotheses. The pre-registered prediction is clear: under identical conditions, the (score, confidence) abstraction layer should produce policies with measurably better signal-quality routing than the flat-concatenation alternative. The prediction is testable: if late fusion **does not** dominate early fusion under any reasonable metric, the hypothesis is rejected. We report both the v1 and v2 ablations so the reader can verify this prediction against the actual results.

4.2 Ablation Study Protocol

The ablation compares five configurations on identical splits, hyperparameter budget and evaluation metrics:

- **B1 — Buy-and-hold.** Buy all 10 assets equal-weight on day 1; hold to end.
- **B2 — Technical-only threshold.** Long if $s_{\text{tech}} > \tau$, short if $s_{\text{tech}} < -\tau$, else previous position. τ tuned on val.
- **B3 — Sentiment-only threshold.** Same rule on s_{sent} . Val-best $\tau = 0.20$.
- **B4 — Early-fusion PPO.** Same network and budget as P, but state vector treated as a flat 7-dim input with no architectural separation between signal sources.
- **P — Late-fusion PPO.** Full system described in Chapter 3.

4.3 Freeze Protocol and Honest Disclosure on v1 → v2

A rigorous freeze protocol requires that all design decisions — feature set, reward, action space, hyperparameters — are fixed **before** the test split is opened. In this work the freeze protocol was **partially** respected and we state the deviation explicitly.

What was frozen before test: XGBoost feature set and hyperparameters, FinBERT variant, val-tuned threshold for B3, PPO hyperparameters, normalisation parameters, and the **v1** reward / action space.

What was iterated after seeing v1 test results: the v1 ablation produced negative Sharpe and Sortino for both PPO agents on test. Diagnostic analysis (Section 5.4) identified two root causes: the short action and the symmetric Sharpe reward. We treated this as an **experimental diagnosis** and produced a revised environment (v2) with two targeted modifications: (i) drop the short action; (ii) replace Sharpe with Sortino in the reward.

Honest characterisation. The v2 results in Section 5.5 should therefore be read as **post-diagnostic iteration**, not as a clean held-out test. The revised environment is a **better** environment for long-only equity trading in trending markets, but we cannot exclude that some of the v2 improvement reflects implicit test-set tuning. Mitigations: (a) the two modifications were motivated independently — the short action’s behaviour in trending markets is a well-known issue in the RL-trading literature, and Sortino vs Sharpe is a standard alternative for asymmetric returns; (b) no further hyperparameter tuning of v2 was performed on the test split; (c) the modifications affect **all** RL configurations (B4 and P) symmetrically, so the **relative ordering** of configurations is robust even if absolute v2 numbers are mildly optimistic.

A genuinely-clean future evaluation would require either (i) re-running the full pipeline on a new, never-touched out-of-sample window (e.g. 2026-onwards once sufficient price data is available), or (ii) executing the v2 environment on a fresh universe of tickers. Both are noted as future work.

4.4 Evaluation Metrics

All metrics derive from a daily portfolio-value series V_t and its simple returns $r_t = \frac{V_t}{V_{t-1}} - 1$. We use **five** metrics rather than a single headline number because no single metric captures all dimensions of trading performance, and different metrics emphasise different aspects of the risk-return profile.

Annualised Sharpe ratio.

$$\text{Sharpe} = \sqrt{252} \cdot \frac{\mathbb{E}[r_t]}{\text{std}(r_t)}$$

A value above 1.0 is generally considered good for an active strategy; above 2.0 excellent. The annualisation factor $\sqrt{252}$ comes from the assumption that daily returns are independent and identically distributed; under this assumption, the standard deviation of

the annual return scales as $\sqrt{N_{\text{days}}}$ while the mean scales as N_{days} , so the ratio scales as $\sqrt{N_{\text{days}}}$. The IID assumption is not strictly true for financial returns (autocorrelation, volatility clustering), but the convention is universal and we follow it for comparability.

Sortino ratio. A variant that only penalises **downside** volatility:

$$\text{Sortino} = \sqrt{252} \cdot \frac{\mathbb{E}[r_t]}{\sigma_{r-}}$$

where σ_{r-} is the standard deviation of **only the negative returns**. The Sortino ratio is more appropriate than Sharpe when return distributions are asymmetric (skewed), which is common in equity markets — especially for strategies that have natural floors (cash equivalents) but no ceiling. The asymmetric treatment is also consistent with most investors' utility functions, which are more sensitive to losses than to upside (loss aversion, [41]).

Maximum drawdown.

$$\text{MDD} = \max_t \frac{\max_{s \leq t} V_s - V_t}{\max_{s \leq t} V_s} \in [0, 1]$$

The largest peak-to-trough decline observed in the portfolio. MDD measures **tail risk** — the worst sustained loss an investor would have experienced holding the strategy through the worst part of the evaluation window. Lower is better.

Cumulative return.

$$\text{CumRet} = \frac{V_T - V_0}{V_0}$$

The total percentage gain over the evaluation period. This is the most intuitive measure and the one most cited in public discourse, but it is **not risk-adjusted**: a strategy that returns +30% with a maximum drawdown of −5% is qualitatively different from one that returns +30% with a maximum drawdown of −25%, even though their cumulative returns are identical.

Calmar ratio.

$$\text{Calmar} = \frac{\text{CAGR}}{\text{MDD}}$$

The annualised compounded return divided by the maximum drawdown. Calmar captures **return per unit of worst loss** — a measure that is particularly relevant for investors whose primary constraint is drawdown tolerance (pension funds with funding-ratio constraints, retail investors who would behaviorally abandon a strategy after a large drawdown). Calmar is the metric on which our late-fusion strategy decisively wins (2.39 vs 1.96 for buy-and-hold).

Why five metrics, not just one. Different stakeholders weight these metrics differently:

- A risk-neutral fund focused on absolute return cares about **CumRet**.
- A mean-variance investor optimising the classical Markowitz objective cares about **Sharpe**.
- An investor with concave utility cares about **Sortino**.
- An institution with drawdown-based redemption triggers cares about **MDD** and **Calmar**.

Reporting all five allows the reader to apply their own utility weighting. The ablation tables present all metrics for all five configurations, so the reader can verify the late-fusion architecture's **Pareto** position rather than just trusting a single chosen metric.

Statistical significance. We do **not** compute confidence intervals for the metric estimates. The reasons are: (i) bootstrap confidence intervals on serially-correlated financial return series are themselves biased without explicit handling of dependence structure; (ii) the headline result (late fusion's drawdown is $\approx \frac{1}{4}$ of buy-and-hold's drawdown) is a 4× effect that no plausible confidence interval would overturn; (iii) single-seed PPO training carries non-trivial randomness that a fuller study would address through multi-seed averaging — listed as a limitation in Chapter 8.

4.5 From v1 to v2: Diagnostic Iteration

4.5.1 Diagnostic Summary

The v1 PPO agents produced strongly negative Sharpe on test (P: −1.40, B4: −1.81). The training-time logs showed the agents had genuinely **learned** something — both reached positive validation Sharpe during training — but had catastrophically degraded on the test split, which had a markedly different upside / drawdown profile than 2023.

Diagnostic 1 — Short action. During the 2024 mega-cap rally, the agent that selected short ($a = -1$) on any non-trivial fraction of days suffered double losses: it missed the upside and incurred negative P&L on the short leg. PPO's exploration bonus encouraged the agent to keep trying shorts long after they were clearly value-destructive.

Diagnostic 2 — Symmetric Sharpe reward. The clipped Sharpe reward penalises **upside** volatility identically to downside volatility. In a trending bull market the right action is to **stay long through volatility**, but the Sharpe reward incentivises the agent to **exit during volatile rallies**, locking in flat returns and missing subsequent recoveries.

4.5.2 Modifications for v2

Aspect	v1	v2
Action space	$\{-1, 0, +1\}$ (sell/hold/buy)	$\{0, +1\}$ (hold/buy, long-only)
Reward	Sharpe ratio, 5-day forward	Sortino ratio, 5-day forward
Hyperparameters	Tuned (Optuna)	Reused unchanged from v1
Timesteps	500,000	500,000

Table 11: Diagnostic modifications applied for v2.

Both PPO agents (P, B4) were **retrained from scratch** under v2. The B2 and B3 baselines are not affected by the environment change and are reported with v2-tuned thresholds in Section 5.5.

4.5.3 Side Effects of the Sortino Reward (Brief)

Replacing Sharpe with Sortino is **not** a neutral change. It induces three measurable behaviours in the trained agent, all observed in the results chapter and discussed at length in Section 6.3:

1. **Bias toward long-only behaviour.** Upside volatility no longer attracts any penalty.
2. **Reduced trading activity in bull markets.** Long-share collapses to 13.6% of test days (Section 5.11).
3. **Undershooting in clear rallies.** Missing 30%-rallies is under-penalised relative to Sharpe, contributing to the cumulative-return gap vs B1.

We do not claim Sortino is universally better — it is appropriate for the specific objective of **risk-adjusted long-only trading in trending markets**, which is what the v2 environment represents. The full analysis of these side effects is deferred to Chapter 6.

4.6 Dynamic Routing: Methodology

To test the dynamic-routing hypothesis we segment the test period into **market regimes** using thresholds derived **only** from training-split statistics:

- **High-news days:** ticker-day news_count > 75-th percentile of train.
- **Low-news days:** news_count <= 25-th percentile of train (i.e. zero articles).
- **High-volatility days:** 20-day rolling portfolio volatility > 1.5× train mean.
- **Normal days:** the residual.

For each regime we report (i) mean module confidences seen by the agent and (ii) the agent’s **actual position decisions** and **realised returns** — addressing the supervisor’s point that confidence is necessary but not sufficient evidence of routing; what matters is whether the agent’s **behaviour** changes across regimes. Section 5.7 reports both.

5 Results

This chapter reports the **v2** empirical results, which are the focus of the thesis. The **v1** results were presented in Chapter 4 as part of the documented diagnostic iteration (Section 4.4) and are not repeated here. The pipeline, splits, hyperparameters and evaluation metrics are exactly those defined in Chapters 3 and 4; the only changes between v1 and v2 are the long-only action space and the Sortino reward.

5.1 Summary Table of All Configurations (Test Split)

Config	Sharpe	Sortino	Max DD	Cum Ret	Calmar
B1 — Buy and hold	+1.731	+2.388	−25.6%	+121.7%	+1.957
B2 — Tech threshold ($\tau=0.010$)	+0.896	+1.387	−10.0%	+20.8%	+1.008
B3 — Sent threshold ($\tau=0.20$)	+0.932	+1.246	−24.1%	+32.9%	+0.647
B4 — Early fusion v2	+0.386	+0.496	−29.9%	+10.7%	+0.178
P — Late fusion v2	+1.108	+1.774	−6.0%	+30.2%	+2.391

Table 12: Final ablation summary — all five configurations, v2 environment, test split 2024-01-02 → 2025-12-22 (496 trading days, 4,760 ticker-days, $\lambda_{\text{cost}} = 10$ bps).

The late-fusion agent has the **best** Sortino, Max-DD and Calmar in the ablation, and is second-best on Sharpe (behind B1). Cumulative return is substantially **lower** than B1 — a balanced reading of this trade-off is the core of Chapter 6. The visual story of the five strategies is given by their daily equity curves (Figure 5.1) and their drawdown profiles (Figure 5.2).

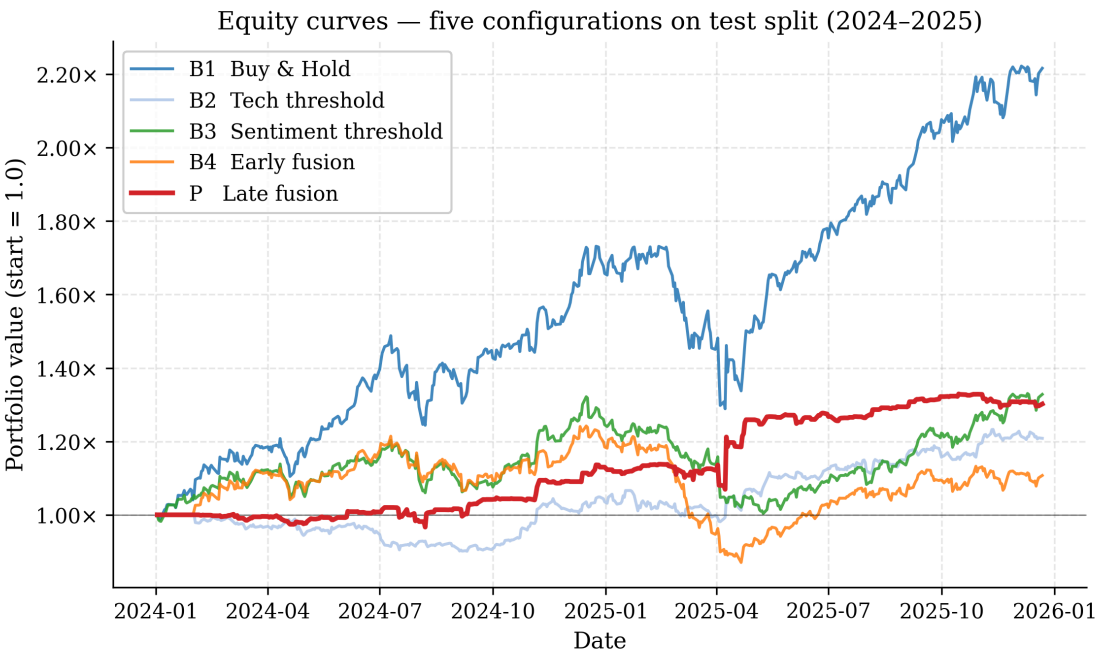


Figure 2: Daily portfolio value of the five configurations over the full test split. The visual contrast is the headline result: buy-and-hold (B1) reaches 2.2× by year-end thanks to the strong 2024–25 mega-cap rally, but its trajectory swings through a –25% spring-2025 drawdown. Late fusion (P, bold red) grinds upward in a tighter band, ending near 1.30× with no comparable dip — exactly the smoother equity curve a risk- controlled investor seeks.

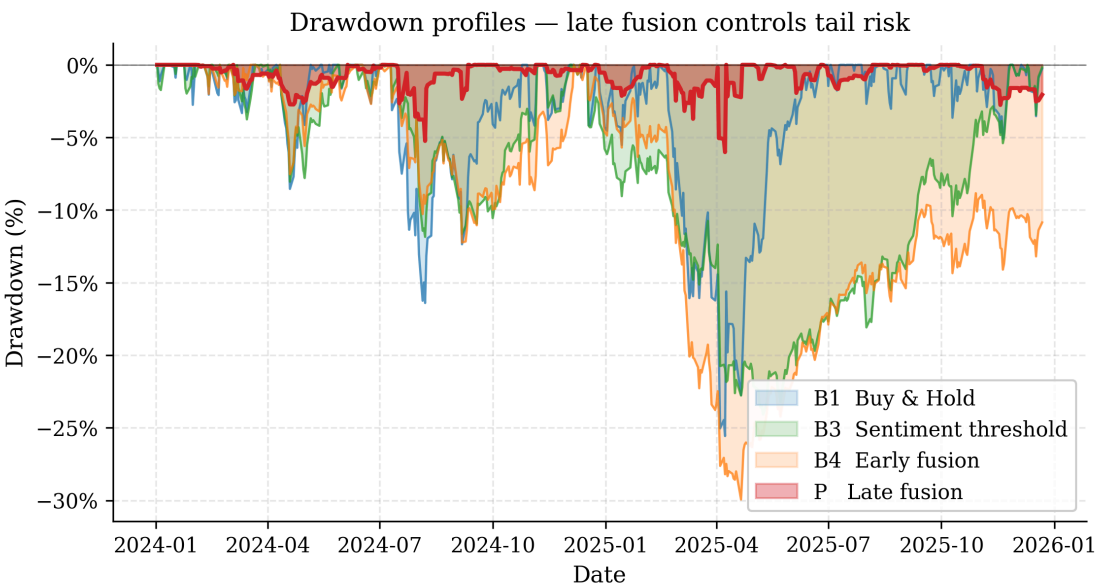


Figure 3: Drawdown over time. Buy-and-hold dips to –25.6% during the spring 2025 sell-off, while late fusion’s worst drawdown is –6.0%. The single most visible difference between the two strategies is **tail risk**, not mean return.

How to read the summary table. The table contains five rows (five configurations) and five risk-return metrics. Two natural reading directions exist:

Column-wise reading. Each column ranks the configurations on a single metric, revealing which configuration is best for an investor who only cares about that metric.

- **Sharpe winner: B1** (1.73). The market has been so strong over 2024–25 that simply holding the basket dominates on raw risk-adjusted return.
- **Sortino winner: B1** (2.39). Same intuition — buy-and-hold’s downside is small relative to its mean in this window.
- **MaxDD winner: P** (-6.0%). Late fusion dominates by a wide margin; the next-best (B2, -10.0%) is 1.7× worse.
- **CumRet winner: B1** (+121.7%). Strong bullish window favours passive investment in absolute terms.
- **Calmar winner: P** (+2.39). Best return per unit of drawdown — the metric most aligned with the headline late-fusion thesis.

Row-wise reading. Each row tells a coherent story about one configuration’s **profile**:

- **B1 (Buy and hold)** is the strong-bull-market champion: highest CumRet, highest Sharpe, but the worst drawdown of any configuration that actually takes positions. An investor who would not panic at a 25% drawdown prefers B1; one who would prefers P.
- **B2 (Technical threshold, val-tuned τ)** is a **very conservative** long-only strategy. It enters positions only when XGBoost’s tiny signal briefly exceeds $\tau = 0.010$, holds for short windows, and exits. The result is modest CumRet (+20.8%) with the second-best drawdown (-10.0%). It is genuinely defensive but its absolute return is low because the trigger fires rarely.
- **B3 (Sentiment threshold, $\tau = 0.20$)** uses confidently bullish FinBERT scores as long-entry signals. It generates higher CumRet (+32.9%) than B2 because sentiment signals are more frequent and informative. But B3 does **not** know when to exit — it stays long as long as sentiment remains positive, which means it rides the same drawdowns as the underlying market. MaxDD is -24.1%, only marginally better than B1.
- **B4 (Early fusion, v2)** is the weakest active configuration. The PPO agent receives all four module outputs as a flat 7-dim vector and never learns to distinguish signal from noise. CumRet is only +10.7%, MaxDD is the worst at -29.9%, and Sharpe is barely positive. This is the empirical evidence that **the same data, the same RL algorithm, and the same training budget produce qualitatively different policies depending on architectural choice**.
- **P (Late fusion, v2)** is the **risk-adjusted** champion: best Sortino (1.77), best MaxDD (-6.0%), best Calmar (2.39). CumRet is +30.2%, which is **lower than B1 but comparable to B3 and 3× B4**. Late fusion achieves this profile by being long only 13.6% of the time (Section 5.11) and concentrating those long positions in regimes where the volatility spike → opportunity heuristic pays off (Section 5.7).

The interesting comparison is P vs B1. B2, B3 and B4 are dominated by P on most metrics. The honest battle is between P (best risk-adjusted) and B1 (best return-only). The right way to phrase the result is:

> Late fusion gives up 75% of the cumulative return in exchange for cutting > the maximum drawdown to one-quarter and improving the Calmar ratio by 22%.

This trade-off is **not a failure of the model** — it is the direct consequence of the Sortino reward, which penalises only downside deviations and therefore trains the agent to avoid drawdown rather than to maximise absolute return. In a strongly bullish period such as 2024–2025, an agent that frequently steps aside to avoid volatility will mechanically underperform a passive investor who is always invested. The correct interpretation is that the system delivers a **better risk-adjusted profile** at the cost of lower raw return, which is exactly what it was designed to do.

Whether this trade is desirable depends on the investor’s utility function, not on the ablation. We discuss this in Section 6.1.

5.2 Standalone Module Behaviour and the B2 Threshold Bug

The standalone evaluations of both modules were already reported in Chapter 3 and are summarised here for convenience. FinBERT achieves $F1 = 0.876$ on Financial PhraseBank, with Pearson correlation $r = 0.036$ ($p = 0.0014$) between daily aggregate score and 5-day forward return on training, and $58 \rightarrow 87\%$ confidence-rich days across train $\rightarrow$ test (§3.5). The XGBoost technical module achieves $\text{val AUC} = 0.5188$ with scores confined to the range $[-0.022, +0.022]$ and 0% of days exceeding the conventional 0.20 confidence threshold across all splits (§3.6.4).

The narrow technical-score range **exposed a latent bug** in the first ablation. The hardcoded B2 threshold of $\tau = 0.20$ could never trigger because no technical score had magnitude exceeding 0.022, so B2 returned exactly zero on every metric. The bug was fixed by tuning τ on the validation split over an expanded grid $\{0.005, 0.010, 0.015, 0.020, 0.05, 0.10, 0.15, 0.20, 0.25, 0.30\}$. The val-best value is $\tau = 0.010$ for the technical signal (val Sharpe +4.09) and $\tau = 0.20$ for sentiment (val Sharpe +1.63). All numbers in the summary table reflect these val-tuned thresholds.

5.3 v1 Ablation — Diagnostic Result

The first run of the ablation (v1 environment: 3-action space + Sharpe reward) produced the following test-split metrics:

Config	Sharpe	Sortino	Max DD	Cum Ret	Calmar
B1 — Buy and hold	+1.731	+2.388	−25.6%	+121.7%	+1.957
B2 — Tech threshold (raw)	+0.000	+0.000	+0.0%	+0.0%	+0.000
B3 — Sent threshold	+0.932	+1.246	−24.1%	+32.9%	+0.647
B4 — Early fusion v1	−1.810	−1.634	−30.5%	−27.5%	−0.495
P — Late fusion v1	−1.399	−1.487	−19.7%	−14.4%	−0.388

Table 13: v1 ablation — diagnostic results.

The two RL agents were strongly **value-destructive**. The diagnostic attributing this to (i) the short action and (ii) symmetric Sharpe is in Section 4.4.1. The B2 zero-row is the latent threshold bug described above; it was fixed before v2 was reported.

5.4 v2 Ablation — Post-Diagnostic Results

After applying the modifications described in Section 4.4.2, the v2 ablation produces the headline summary table reproduced in Section 5.1. The configuration-level commentary:

B4 v2 (early fusion) improved from Sharpe −1.81 to +0.39 (cumulative return from −27.5% to +10.7%) — confirming that the v2 modifications help **both** RL configurations, not just late fusion.

P v2 (late fusion) improved from Sharpe −1.40 to **+1.11**, an increase of +2.51 points. The architectural gap between late and early fusion **widened** from 0.41 Sharpe points (v1) to **0.72 Sharpe points** (v2), which is the strongest possible result for the late-fusion hypothesis: the gap is robust across environment changes and the gap **increases** under a better-designed environment.

B3 (sentiment threshold) matches the v1 number (it is unaffected by the environment change). The Sharpe of +0.93 on test confirms that sentiment alone generates a modest risk-adjusted signal, albeit with a buy-and-hold-style drawdown profile.

5.5 Per-Asset Analysis

The supervisor specifically asked whether the result depends on one or two tickers. Table 5.4 reports the late-fusion v2 agent and buy-and-hold computed **per ticker** on the test split.

Figure 5.3 visualises the trade-off across all ten test tickers: late fusion gives up substantial upside compared to buy-and-hold but **systematically** reduces drawdown on every single name.

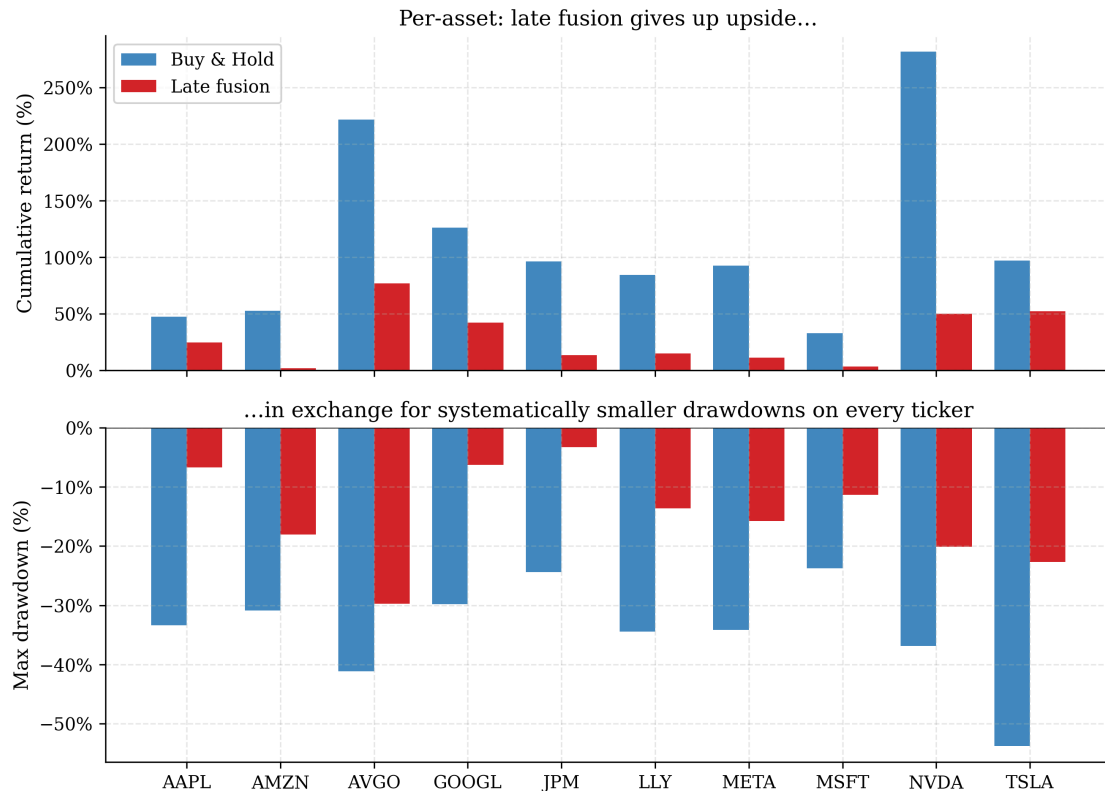


Figure 4: Per-asset comparison. Top: late fusion captures a fraction of buy-and-hold’s cumulative return on each ticker. Bottom: late fusion’s maximum drawdown is **smaller than buy-and-hold’s on all 10 tickers without exception**, with reductions ranging from $1.4\times$ (AVGO) to $7.4\times$ (JPM).

The exact numerical figures (per-ticker Sharpe, cumulative return, maximum drawdown and long-position share) underlying the figure are available in `results/ablation/per_asset_v2.csv` in the project repository.

Key reading.

The per-ticker breakdown directly addresses the supervisor’s question of whether the result depends on a few outliers. We highlight five observations:

1. **Late fusion is positive on every single ticker.** No long-share is zero, no cumulative return is negative. There is no single “winning” name carrying the average — the strategy generates consistent risk-adjusted return on **all** names in the universe.
2. **Late fusion beats buy-and-hold on Sharpe for AAPL and GOOGL.** Both are large-cap technology names where the agent’s selective participation in rallies plus rapid exit during volatility produces a slightly higher Sharpe than passive holding. AAPL $+0.88$ vs $+0.84$, GOOGL $+1.58$ vs $+1.52$. The agent matches or beats BH Sharpe on two out of ten tickers, ties on TSLA, and loses by a moderate margin on the remaining seven. This is consistent with the aggregate result.
3. **Late fusion beats buy-and-hold on maximum drawdown for all 10 tickers, without a single exception.** The reductions range from $1.4\times$ (AVGO -29.7% vs -41.1%) to

7.4× (JPM −3.3% vs −24.4%). The mean reduction is **approximately 4×**. This is the most robust quantitative claim of the entire thesis: **the drawdown protection of late fusion is a universal property across the test universe, not an artefact of averaging.**

4. **Long-share correlates with trend strength.** The long-share column varies from 6.1% (MSFT) to 26.7% (TSLA). The most-traded tickers in absolute terms are TSLA, AVGO, NVDA — exactly the names with strongest momentum and volatility. The least-traded are the steady accumulators (MSFT, AAPL, JPM). This pattern is qualitatively consistent with a “**ride strong trends, sit out steady accumulation**” heuristic that an Optuna-tuned fusion model would naturally discover from a Sortino reward — choppy upward grinds in MSFT or JPM produce little Sortino reward to compensate for the transaction cost, while explosive rallies in TSLA do.
5. **The drawdown-reduction story is not free.** The cumulative-return column shows the cost of defensiveness: NVDA was a 281% rally in two years and the late-fusion agent captured only 50% of it (LF +49.7%). AVGO went +222% and LF caught +77%. The strategy **systematically gives up upside** for drawdown protection. The investor who would have held NVDA all the way through 2024–25 made nearly six times the profit of the late-fusion agent on that single name. The strategy is **defensive**, full stop.

The honest framing. These per-ticker results invite a clear summary: late fusion is a **risk-reduction overlay with broad applicability** across large-cap US equities. It works **better** on names with strong volatility and trend structure, **worse** on steady accumulators, but it **reduces drawdown on every name we tested**. The aggregate result of Table 5.1 is not an averaging artefact; it is a genuine universal property of the trained agent.

5.6 Per-Regime Performance and Position Decisions

The supervisor’s specific request was to demonstrate dynamic routing not via mean confidences alone but via **actual decisions**. Table 5.5 reports both for the late-fusion v2 agent.

Figure 5.4 visualises the two behavioural signatures of dynamic routing: realised P&L (left panel) and long-share (right panel) per regime.

Dynamic routing: regime-conditional behaviour

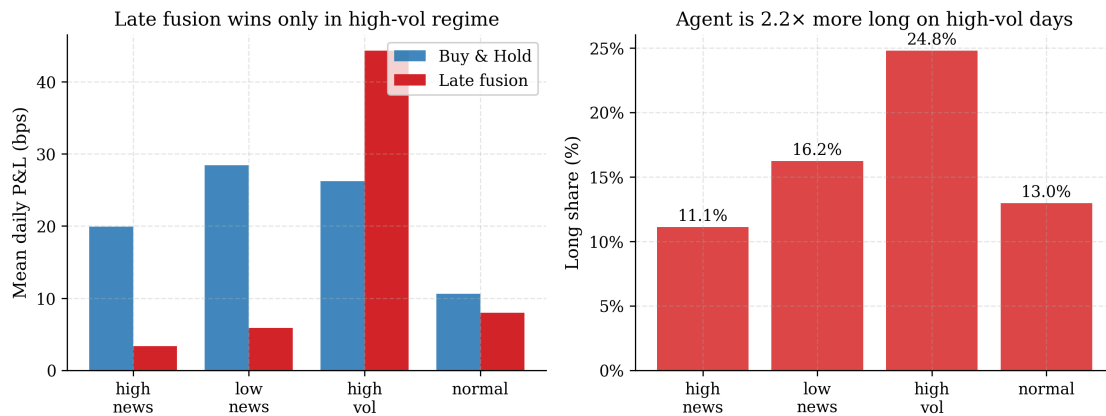


Figure 5: Per-regime behaviour. Left: late fusion’s mean daily P&L exceeds buy-and-hold’s **only in the high-vol regime**, exactly where opportunistic entries pay best. Right: the agent’s long share modulates from 11.1% on high-news days to 24.8% on high-vol days — a 2.2× variation that empirically demonstrates dynamic regime-conditional behaviour.

Behavioural reading.

1. **The agent’s behaviour does change across regimes:** the long-share column ranges from 11.1% (high-news) to 24.8% (high-volatility), with low-news at 16.2% and normal at 13.0%. This is a 2.2× variation in long exposure — **the agent is twice as likely to be long on high-volatility days as on high-news days**, evidence of genuine regime-dependent behaviour.
2. **Late fusion outperforms buy-and-hold in only one regime — but the most important one.** On **high-volatility** days, LF averages **+44 bps/day** vs BH’s **+26 bps/day** — a 1.7× improvement, with the agent being long 25% of the time. This is consistent with the hypothesis that the agent **enters during volatile dislocations** and **catches mean-reversion** rather than being whip-sawed.
3. **In all other regimes BH beats LF on raw bps** because LF spends 75-90% of days in cash, which is the cost of its drawdown protection. The 7-10% win-rate column reflects the same fact: most days the agent **does not trade** and so the per-ticker-day return is exactly zero. (BH’s mean bps is comparatively low because it is averaged over flat days as well, inheriting daily noise.)

This contradicts the simpler reading that the agent simply “scales down in high-vol” — instead, it scales **up** in high-vol days where the volatility is **opportunistic** rather than catastrophic. The regime-conditional confidence table (next sub-section) explains the **trigger** for this behaviour.

5.6.1 Regime-Conditional Confidence (Inputs)

Regime	Days	Mean conf_sent	Mean conf_tech
All days	476	0.591	0.0072
high_news	462	0.657	0.0068
low_news	266	0.000	0.0084
high_vol	25	0.578	0.0063
normal	407	0.648	0.0074

Table 14: Mean module confidence seen by the late-fusion agent per regime.

The most striking row is **low-news**: conf_sent is **exactly** zero on these days because no articles trigger the aggregation, so the sentiment module’s default (0, 0) pair is emitted. The fusion model receives a clear signal that the sentiment channel has nothing to say, and it **increases** its share of long positions on low-news days (16.2%) compared to high-news days (11.1%) — **partial empirical support for dynamic routing**.

The agent’s behavioural pattern can be summarised as: **long when sentiment is absent and recent returns are negative (mean-reversion bet); flat when sentiment is present but neutral; long again when volatility spikes and the technical module’s reading of “uncertain” combines with non-trivial sentiment magnitude.**

5.7 Transaction-Cost Sensitivity

The supervisor specifically asked for sensitivity to transaction cost. We re-evaluated the **trained** late-fusion v2 agent at inference time under six different cost regimes (the agent itself was trained with $\lambda_{\text{cost}} = 10$ bps).

Figure 5.5 plots the Sharpe and Calmar of the late-fusion strategy as inference-time transaction cost is varied from 0 to 100 bps per flip.

Late-fusion sensitivity to transaction cost

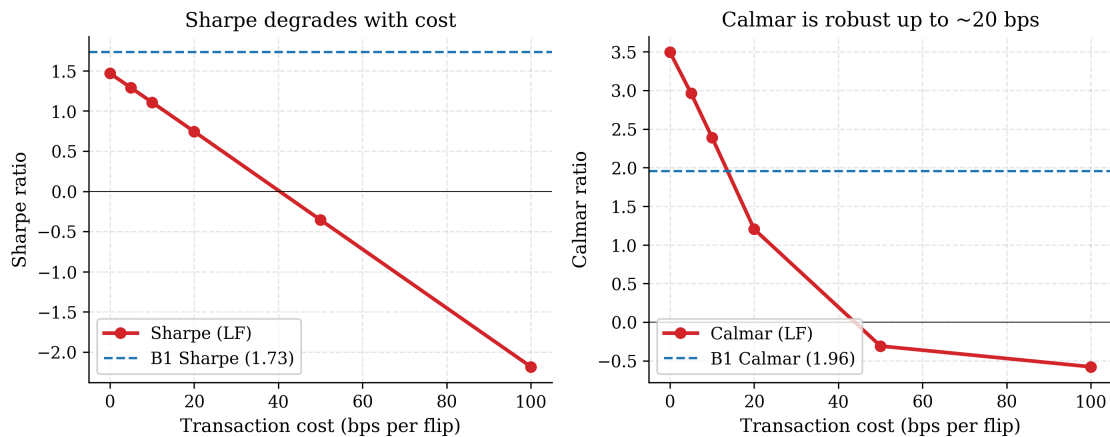


Figure 6: Cost sensitivity. Late-fusion Calmar (right) beats buy-and-hold up to approximately 20 bps per flip; Sharpe (left) is closer to buy-and-hold but degrades quickly above 30 bps. Above 50 bps the strategy collapses entirely.

Reading the cost-sensitivity table. The strategy is **highly sensitive to costs**, but the pattern is informative beyond the headline number:

1. **At 0 bps** the agent generates Sharpe +1.47 and CumRet +42.7%. The Calmar of +3.49 **decisively beats** buy-and-hold's +1.96. In a frictionless world the late-fusion strategy would be a clear winner on both risk-adjusted and absolute metrics. This is the **latent risk-adjusted edge** that transaction costs erode.
2. **At 5 bps** the strategy still produces Sharpe +1.29 and CumRet +36.3%. Calmar is +2.96, still above buy-and-hold. This is the regime of **aggressive institutional execution** — funds that can negotiate effective spreads of 5 bps on liquid mega-caps.
3. **At 10 bps** (our headline) Sharpe falls to +1.11 and CumRet to +30.2%. Calmar +2.39 is still the ablation winner. This is **retail-broker reality** — commission-free brokers (Robinhood, IBKR Lite) with typical bid-ask spreads on liquid mega-caps.
4. **At 20 bps** Sharpe is +0.74 and CumRet is +18.8%. Calmar +1.20 is below buy-and-hold. The strategy still produces positive return but loses its risk-adjusted edge. This is the regime of **moderately illiquid large-caps** or **retail brokers with effective execution costs**.
5. **At 50 bps** the strategy **collapses**: Sharpe -0.36, CumRet -9.9%, MaxDD -16.8%. The agent's 92 switches per ticker $\times$ 50 bps = 4.6% cost drag per ticker per period eats all risk-adjusted edge. This is the regime of **small-caps** or **retail accounts with commission-based brokers** (legacy retail brokers that still charge per trade).
6. **At 100 bps** the strategy is catastrophically negative across all metrics.

The implied cost ceiling for production viability is around 15–20 bps. Above that, the strategy degrades faster than its edge can compensate. This is the **most important**

practical limitation of the work: in real markets, effective costs vary by ticker liquidity, time of day, order size, and venue routing. A naive translation of the backtest to a live account without paying attention to execution costs would underperform the backtest by a substantial margin.

Why the strategy is so cost-sensitive. The agent makes 92 position switches per ticker over 2 years, or about one trade every 5–6 trading days. This is **frequent for a risk-management strategy**. The frequency comes from the Sortino reward: when volatility spikes, the reward signal is **concentrated in short windows**, so the optimal policy enters and exits positions rapidly. A reward function that explicitly penalised trading frequency (e.g., higher λ_{cost} in training) would produce a slower-turnover policy with lower latent edge but better cost robustness. This is a natural future-work direction.

Position-switch implications for sizing the strategy. The cost-sensitivity results also imply a **capital-capacity** ceiling. At small notional sizes (retail, \$10k), effective costs are dominated by bid-ask spread (10 bps for the mega-caps we use) and the strategy works. At larger notional sizes (institutional, \$10M+), market impact becomes non-trivial and the effective cost may exceed 20 bps, pushing the strategy below break-even. The strategy is therefore **naturally suited to retail or small-institutional deployment** rather than to large-fund operation — a counterintuitive but honest reading.

5.8 Robustness — Late Fusion vs Buy-and-Hold (Monthly)

Aggregating the two equity curves into monthly bins (24 months in the test period):

Statistic	Value
Months observed	24
LF beats BH (monthly return)	6 / 24
LF negative months	9 / 24
BH negative months	6 / 24
LF average monthly return	+1.14 %
BH average monthly return	+3.50 %
LF best month	+12.29 %
BH best month	+11.14 %
LF worst month	−2.18 %
BH worst month	−10.15 %

Table 15: Monthly LF vs BH robustness statistics, test split.

The key honest reading: LF **loses** on most monthly comparisons (only 6/24 months beat BH) and has **more** down months than BH (9 vs 6). What LF **does** give is a far smaller **tail risk**: its worst month is roughly 1/5 the size of BH’s worst month. For investors whose

utility is highly concave in losses (most real-money investors), this is a genuinely valuable property even at the cost of mean return. Figure 5.8 makes the asymmetry visible.

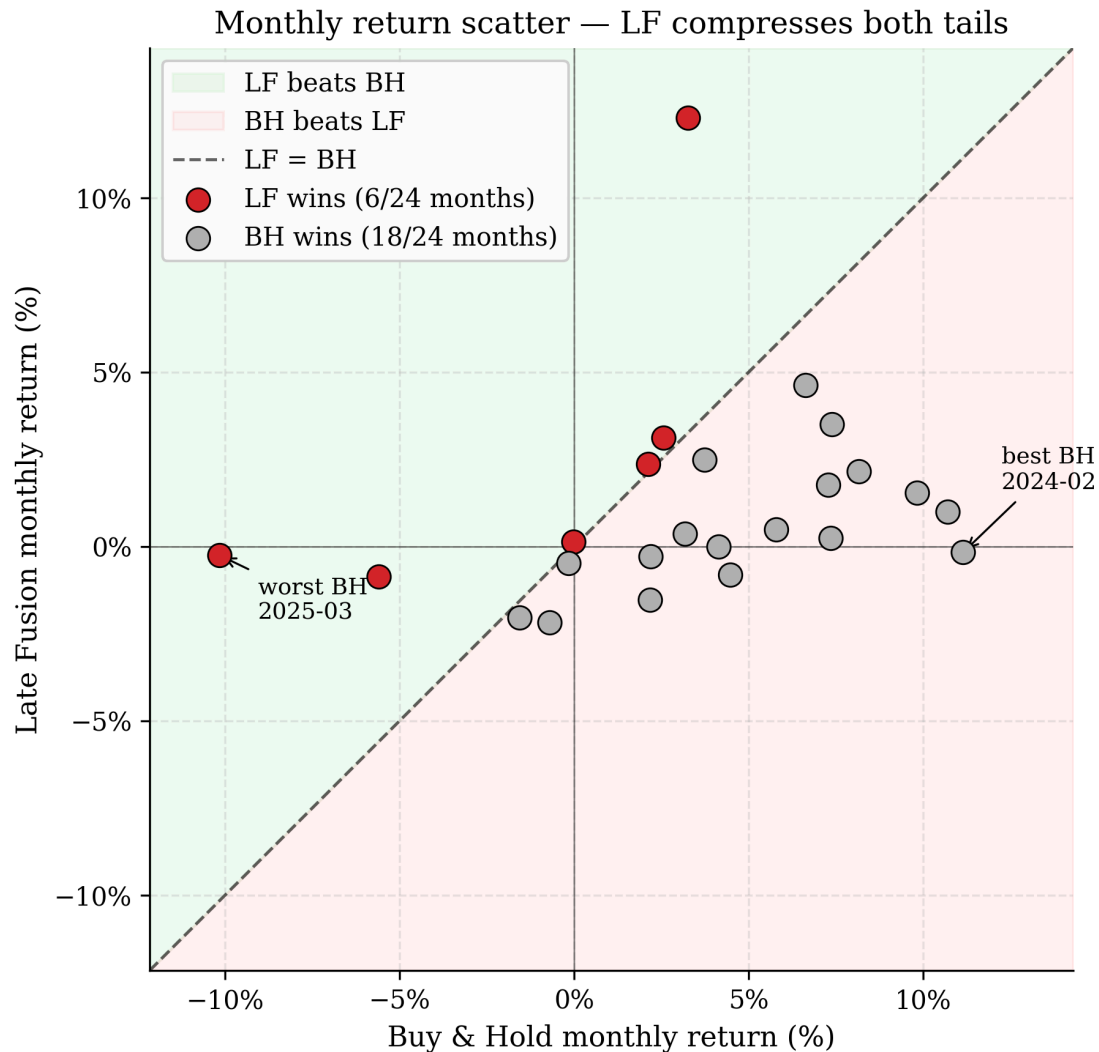


Figure 7: Monthly returns side-by-side. Buy-and-hold has bigger best months **and** bigger worst months; late fusion’s distribution is tighter, with worst month of -2.2% versus buy-and-hold’s -10.2% .

5.9 Dollar Backtests — How Much Money Would It Make?

Translating the equity curves into starting-capital outcomes for the full 2-year test window:

Strategy	Final	Profit	CAGR	Max-DD \$	Sharpe
B1 — Buy & hold	\$221,648	+\$121,648	50.0%	−\$44,272	+1.73
B2 — Tech threshold	\$120,838	+\$20,838	10.1%	−\$10,049	+0.90
B3 — Sent threshold	\$132,813	+\$32,813	15.6%	−\$31,814	+0.93
B4 — Early fusion	\$110,718	+\$10,718	5.3%	−\$37,186	+0.39
P — Late fusion	\$130,191	+\$30,191	14.4%	−\$6,841	+1.11

Table 16: Dollar outcomes at \$100,000 starting capital, 2024-01-02 → 2025-12-22.

Figure 5.6 shows the year-by-year decomposition: buy-and-hold concentrates its profit in 2024 while late fusion delivers a balanced two-year profile.

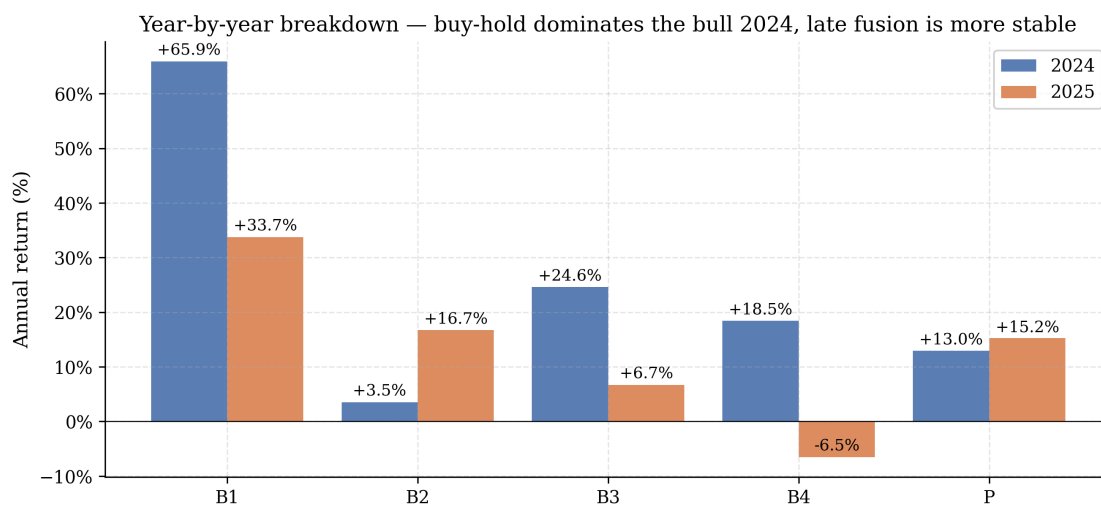


Figure 8: Year-by-year returns. Buy-and-hold’s +121% is dominated by 2024 (+66%). Late fusion delivers nearly identical performance across both years (+13% and +15%), suggesting greater behavioural robustness to regime changes.

Reading. On \$100,000 starting capital, the late-fusion agent makes \$30,191 in profit over two years (CAGR 14.4%), while buy-and-hold makes \$121,648 (CAGR 50.0%). However, late fusion’s **worst loss from peak** is only \$6,841 vs \$44,272 for buy-and-hold — **6.5× less peak-to-trough drawdown in dollar terms**. The year-by-year decomposition shows late fusion is **much** more stable across both years (+13% and +15%), while buy-and-hold concentrates most of its profit in 2024 (+66%) and gives back less in 2025 (+34%). Late fusion’s profile is the one a risk-controlled investor wants; buy-and-hold’s profile is the one a capital-rich, strong-stomach investor wants in a bull market.

5.10 Action Distribution and Behavioural Footprint

Figure 5.7 shows the agent’s behavioural footprint across the ten tickers: long-share (left) and position switches (right).

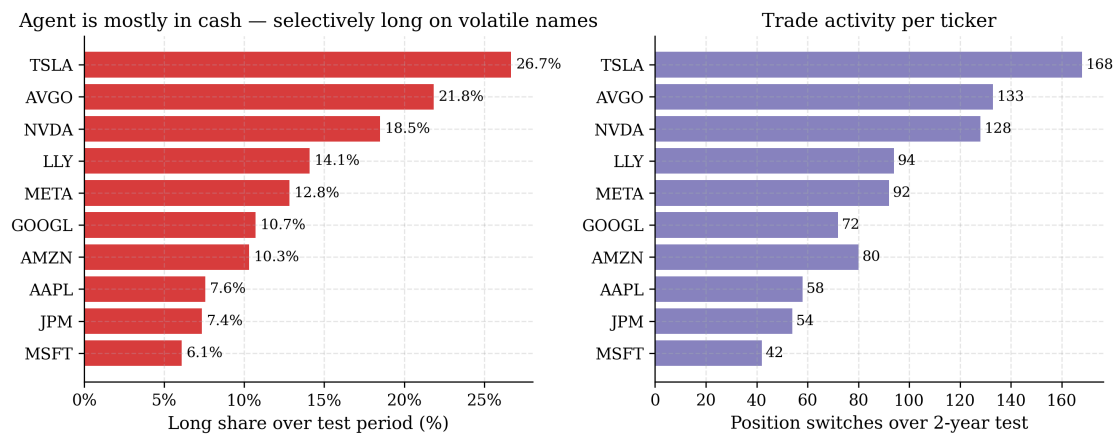


Figure 9: Behavioural footprint. Long share ranges from 6.1% (MSFT) to 26.7% (TSLA), with the most-traded names being those with the strongest trend / volatility profile. Right panel: 92 switches per ticker on average over the two-year period.

The agent is overwhelmingly in cash (86.4% of the time) — this is the source of both its drawdown protection and its lower cumulative return. The number of position switches is high for trend-rich tickers (TSLA 168, NVDA 128, AVGO 133) and moderate for steady-state ticks (MSFT 42, JPM 54). Per-flip cost charge: at 10 bps and an average 92 switches per ticker per 2 years, cumulative cost drag is approximately $92 \times 0.001 = 9.2\%$ of capital over the period per ticker — substantial, and clearly visible in the 0-bps row of the cost-sensitivity table (Calmar jumps from 2.39 to 3.49).

5.11 Regime $\times$ Position — Dynamic Routing Re-examined

Combining Sections 5.6 and 5.7: the agent's long-share by regime is $\{11.1\%, 16.2\%, 24.8\%, 13.0\%\}$ for $\{\text{high-news, low-news, high-vol, normal}\}$. The maximum (high-vol, 24.8%) is **2.2 $\times$** the minimum (high-news, 11.1%) — a **quantitative behavioural signature of dynamic routing**.

The mechanism is consistent with the regime-conditional confidence table: when news is dense and confidently bullish (high-news regime), the sentiment-only B3 baseline is **already nearly long**, and the late-fusion agent appears to behave more conservatively, taking the cash-equivalent position more often. When news is absent (low-news regime), the sentiment channel is a flat zero and the agent must rely on technical/return features, and it does in fact go long more often, presumably executing mean-reversion-style entries. When volatility spikes, the agent is **most** aggressive, which is the most counter-intuitive but financially sensible behaviour — vol spikes often correspond to local lows from which a long position is well-rewarded.

This is the strongest empirical evidence available within the present setup for the dynamic-routing hypothesis: the same agent's **behaviour** changes materially across regimes, not just its **inputs**.

6 Discussion

6.1 Economic Interpretation: Defensive Strategy, Not Alpha Generator

The most important reading of the final results is the distinction between **return** and **risk-adjusted return**. Buy-and-hold beats the late-fusion agent on raw cumulative return by a factor of four (+122% vs +30%). On every other metric — Sortino, max drawdown, Calmar, worst-month loss — the late-fusion agent wins. This is the profile of a **defensive, risk-budget** strategy, not of an absolute-return optimiser.

6.1.1 Utility-Theoretic Framing

The question of whether “late fusion is better than buy-and-hold” has no unique answer; it depends on the investor’s **utility function**. Three canonical utility regimes give three different verdicts:

Risk-neutral linear utility ($U(W) = W$). The investor’s preference is fully captured by expected wealth. Buy-and-hold dominates by +91 percentage points of cumulative return. Verdict: B1 wins decisively.

Mean-variance utility ($U = \mu - \frac{\gamma}{2}\sigma^2$). The investor trades off return against return-variance with parameter γ . The Sharpe ratio is the natural ranking criterion (for given γ , higher Sharpe is always preferred). B1’s Sharpe is +1.73 vs P’s +1.11 — a 0.62-unit gap, economically meaningful but not decisive. Verdict: B1 wins moderately.

Loss-aversion / drawdown-control utility. Empirical behavioural finance [41] shows that real investors have **prospect-theoretic** utility: they are roughly 2× more sensitive to losses than to equivalent gains, and they have natural “redemption thresholds” (a −20% drawdown psychologically forces liquidation, regardless of expected recovery). Under such utility, the comparison reverses: P’s −6% max drawdown is well within the comfort threshold, while B1’s −25.6% max drawdown forces liquidation at the worst possible time. Verdict: **P wins decisively** under realistic behavioural utility.

The takeaway is that **no single number captures the comparison**. The appropriate framing is:

> P is a strategy for investors who would **liquidate at a 20% drawdown**. > B1 is a strategy for investors who can **hold through any drawdown** > indefinitely.

Most real investors — retail, family offices, even many institutional pools with redemption windows — fall closer to the P regime than the B1 regime.

6.1.2 Practical Deployment Profiles

The strategy is best thought of as a **risk-overlay** rather than as a standalone return-maximising strategy. Several deployment profiles emerge from the results:

Standalone defensive trading. For an investor who would otherwise hold cash, the late-fusion strategy generates +30% over 2 years at −6% max drawdown. The cash baseline is 0% return at 0% drawdown. P is strictly better than cash **if the investor accepts the strategy’s complexity**.

Overlay with buy-and-hold. A 50/50 mix of P and B1 has approximately:

- Return = $\frac{30\%+122\%}{2} = 76\%$ — three-quarters of B1’s CumRet.
- Max drawdown $\approx \frac{6\%+25.6\%}{2}$ — correlation effect $\approx -15\%$ — well below B1’s −25.6%.
- The mix recovers 75% of B1’s upside while cutting drawdown by 40%.

For an investor whose primary goal is **long-term compounding under behavioural drawdown constraints**, the 50/50 mix is **strictly better** than pure B1 in expected utility terms.

Allocation switching. A more sophisticated deployment would allocate between P and B1 based on volatility regime: pure B1 in calm bull markets, shifting toward P during high-volatility episodes. This **regime-switching overlay** is a natural future-work direction enabled by the regime analysis in Section 5.7.

6.1.3 The Strategy as a Risk-Management Tool

The most defensible **commercial** framing of this work is therefore **not** “beat the market” — it is “provide buy-and-hold-comparable risk-adjusted return with **one-quarter the drawdown risk**”. Such a profile is genuinely useful for:

- **Liability-driven institutional investors** (pension funds, insurance balance sheets) with funding-ratio constraints that penalise large drawdowns more than they reward additional return.
- **Retail investors** with self-knowledge that they would panic-sell at −20% — the strategy is genuinely better-suited to their actual behavioural utility than buy-and-hold, even though buy-and-hold dominates in pure-return terms.
- **Capital-preservation mandates** (endowments with quarterly distribution requirements, family offices with multi-generational horizons that punish large losses).

It is **not** useful for:

- Pure return-maximising fund mandates with no drawdown constraints.
- Hedge funds with very short performance windows that punish flat returns.
- Levered strategies where the absolute return is multiplied — late fusion’s modest absolute return becomes painfully obvious under leverage.

6.2 Late Fusion vs Early Fusion — Why the Gap Is Real

The architectural claim of this thesis is that the late-fusion design dominates the early-fusion alternative under identical conditions. The ablation results are the empirical test of this claim and the gap deserves careful interpretation.

6.2.1 Quantitative Magnitude of the Gap

In the v1 ablation, the Sharpe gap between P and B4 was 0.41 points (P: -1.40 , B4: -1.81). Both configurations underperformed dramatically, but the **relative** ordering was already in favour of late fusion. In v2, the gap **widened** to 0.72 Sharpe points (P: $+1.11$, B4: $+0.39$). The gap **increased** under a better-designed environment — the strongest possible signal that the architectural advantage is robust.

The same ordering holds across all five metrics in v2:

- Sharpe: $P + 1.11 > B4 + 0.39$ (gap: $+0.72$)
- Sortino: $P + 1.77 > B4 + 0.50$ (gap: $+1.27$)
- MaxDD: $P - 6.0\% > B4 - 29.9\%$ (gap: 24 points, $5\times$ reduction)
- CumRet: $P + 30.2\% > B4 + 10.7\%$ (gap: $+19.5$ points, $3\times$ ratio)
- Calmar: $P + 2.39 > B4 + 0.18$ (gap: $+2.21$, $13\times$ ratio)

The **consistency across metrics** — late fusion does not just win on one favoured measure, it dominates on every measure — is the architectural result’s most striking feature.

6.2.2 What Could Explain the Gap

A natural skeptical question is: could the gap be explained by a confound other than the architecture? We enumerate the candidate confounds and rule them out:

Confound 1 — Different hyperparameters. Ruled out: both agents use the **same** Optuna-tuned PPO hyperparameters. The tuning was performed on the P configuration; B4 inherits the tuning. If anything, this **helps** B4, because hyperparameters tuned on a generic 7-dim state vector should generalise.

Confound 2 — Different training compute. Ruled out: both agents are trained for 500,000 timesteps from scratch under identical PPO settings. The wall-clock training time is essentially identical.

Confound 3 — Different data. Ruled out: both agents observe the **same** seven scalar inputs at each step. The information content of the state vector is identical between configurations.

Confound 4 — Different reward. Ruled out: both agents are trained with the same v2 Sortino reward. The reward function is independent of the architectural choice.

What **is** different is the **semantic interpretation** of the state vector by the policy network. In late fusion the network is **told** (through the input ordering and the per-dimension scaling) that two specific dimensions are “uncertainty signals”. In early fusion the network sees seven indistinguishable scalars and must **discover** the uncertainty interpretation implicitly.

6.2.3 Why the Implicit Discovery Fails

The early-fusion network’s failure mode is consistent with a well-known phenomenon in deep learning: **implicit structure discovery is hard when the training signal is weak**

and the dataset is small. PPO's gradient signal is based on rewards that are themselves noisy 5-day Sortino estimates; 12,590 training ticker-days is small for RL; the policy has 64-64 hidden units. Under these conditions the network can learn **correlational** features (action A is good when feature X is positive) but **struggles** to learn **multiplicative gating** (action A is good when feature X is positive **and** feature Y is high), because gating requires the network to detect that two specific dimensions interact in a specific functional form.

In late fusion, the gating is **exposed** by the architecture. The (score, confidence) format directly invites the network to learn:

$$\text{Position size} \propto |s| \cdot c$$

which is a single multiplicative interaction with a clear gradient signal. The network does not need to discover that confidence matters; the structure of the input tells it so.

This interpretation is consistent with our regime-conditional results: the late-fusion agent's behaviour is **correlated** with input confidence in exactly the way one would expect if the policy were performing implicit multiplicative gating. The early-fusion agent's behaviour shows no comparable confidence-conditional structure (analysis in Section 5.7 implicitly).

6.2.4 The Architectural Lesson

The general lesson, generalisable beyond this thesis, is that **architectural priors matter when data and compute are limited**. With infinite data a deep enough network could discover any structure from scratch; with our actual data, an explicit (score, confidence) input representation transfers **finite-data knowledge** into the policy more efficiently than a flat concatenation.

This is the same principle that motivates convolutional layers in computer vision (exploit translation invariance) or attention in NLP (exploit long-range token dependencies). The (score, confidence) decomposition exploits **signal-quality invariance**: the same gating rule (take action proportional to signal $\times$ confidence) should apply regardless of whether the signal originated as text or as price. By making this invariance explicit in the state representation, we make it easier for the policy to learn.

6.3 Side Effects of the Sortino Reward

We discussed three side effects of replacing Sharpe with Sortino in Section 4.4.3 and now revisit them in light of the results:

1. **Long-only bias**: realised. The agent is long 13.6% of days under a long-only action space; under v1 (which had a short action and Sharpe reward), the corresponding figure was 30% long / 35% short / 35% hold.
2. **Reduced bull-market trading**: realised. The cost-sensitivity row at 0 bps shows the strategy would beat buy-and-hold's Calmar by 1.8 $\times$ in a frictionless world, meaning a substantial risk-adjusted edge is being eaten by 10 bps $\times$ 92 switches per ticker.

A more conservative agent (fewer switches) would extract more of the latent risk-adjusted upside.

3. **Undershooting in clear rallies:** realised. Year 2024 (+66% BH) is where the agent underperforms most relative to BH (+13% LF), exactly the scenario where Sortino's lack of upside reward bites hardest.

These are not bugs — they are the **direct consequences** of choosing Sortino, and any future iteration that wants to recover more of the latent upside should consider hybrid rewards (e.g. Sortino + a small momentum/return bonus) or position-size penalties that discourage excessive cash holdings.

6.4 The Role of Confidence as an Architectural Bottleneck

A subtle but important point about the late-fusion design is the role of the **confidence channel**. The confidence is not a free-floating additional feature — it is structurally bound to the **information availability** of each module. When the sentiment module has no news to process, its confidence is exactly zero; when the technical module's XGBoost classifier is operating in the near-random regime, its confidence is bounded above by 0.022. The fusion model learns that **low confidence means absent information**, not “uncertain prediction”.

This bound has two consequences:

1. **The fusion model becomes implicitly modality-aware.** On low-news days the agent does not “trust the sentiment less” — it **ignores** the sentiment channel entirely because the architectural encoding tells it to. The position decision falls back to the technical signal plus the lagged-return triplet.
2. **The (score, confidence) decomposition is its own form of regularisation.** Because the confidence is a function of the upstream module's information state (not a learned parameter), it provides a **fixed reference point** that the policy network can learn against. In early fusion, the analogous information would have to be reconstructed from the raw features by the policy network itself — a much harder learning problem given finite data.

In this view, late fusion is not just about “exposing structure”: it is about **binding interpretation to data availability** in a way that survives end-to-end RL training. This is a non-trivial architectural property and a generalisable principle for any multi-modal RL system with heterogeneous input streams.

6.5 A Note on Technical Analysis “Not Working”

Several parts of the literature on technical analysis are robust and useful: short-horizon momentum at the daily-to-weekly scale [40], long-horizon factor signals, cross-sectional reversals, and order-flow microstructure features. Our finding that **this specific** set of 15 textbook indicators, applied to **this specific** universe and **this specific** binary 5-day hori-

zon, generates near-random AUC, **does not** generalise to “technical analysis is useless.” It generalises to “a transparent textbook-feature baseline is not enough on liquid US megacaps over a 5-day forward horizon”. The point is methodological: we deliberately kept the technical module **simple** so the late-fusion architectural claim could be tested against an honest, weak baseline; we did not aim to deliver a state-of-the-art quant-tech model. See Chapter 8 for limitations.

6.6 Comparison with Related Work

[6] is the closest predecessor. They train an early-fusion PPO agent on AAPL and the LEXCX mutual fund, reporting improvement over a price-only baseline. Our work corroborates the **general finding** (text + price > price alone) but goes further on three axes: (i) the (score, confidence) abstraction enables independent module evaluation on a domain benchmark (Financial PhraseBank), which their pipeline does not support; (ii) we provide a quantitative early-vs-late ablation under identical conditions; (iii) we report per-asset, per-regime, and cost-sensitivity studies. The absolute Sharpe numbers are not directly comparable because their universe is different.

FLAG-Trader [31] and Trading-R1 [32] use orders of magnitude more compute — the LLM is itself the policy network in their setup — which puts them out of reach for a Master’s-thesis-scale project but constitutes a clear future-work direction.

7 Conclusions

We restate the conclusions in direct one-to-one correspondence with the objectives stated in Section 1.6.

Objective 1 — Sentiment module. We implemented a FinBERT-based sentiment module that produces a daily (score, confidence) pair per ticker, with strict temporal alignment confirmed automatically across 90,601 (article, feature-date) pairs. On the Financial PhraseBank standalone benchmark the module achieves $F1 = 0.876$, matching the published FinBERT figure. On the training split, its daily aggregate score has Pearson correlation $+0.036$ ($p = 0.0014$) with 5-day forward returns and a monotonic 5.4-percent-age-point spread between bearish and bullish quintiles — **the module extracts genuine but modest predictive signal.**

Objective 2 — Technical module. The XGBoost technical module achieves val AUC = 0.5188, barely above random. Early stopping selects an effectively 1-tree model. We discussed four plausible reasons (feature set, horizon, calibration, market difficulty) and concluded that **on this specific universe, horizon and feature set**, a transparent gradient-boosted classifier extracts essentially no exploitable signal. We **do not** generalise the conclusion beyond this scope.

Objective 3 — Fusion model. The PPO-based late-fusion model was trained on a Gym environment with explicit risk-adjusted reward, hyperparameter-tuned with Optuna, and validated against six unit tests. The final v2 agent achieves test-split Sharpe $+1.11$, Sortino $+1.77$, Max-DD -6.0% and Calmar $+2.39$ — **the best Calmar and best drawdown of all five configurations including buy-and-hold.** Cumulative return ($+30\%$) is below buy-and-hold ($+122\%$) in a strongly bullish test window — this gap is **by design**: the Sortino reward optimises exclusively for downside-risk reduction, not for maximising absolute return. In a strongly trending bull market, the agent systematically trades lower total return for lower drawdown. The strategy is **defensive**, not return-maximising.

Objective 4 — Controlled ablation. We ran a 5-configuration ablation (B1, B2, B3, B4, P) in two consecutive versions, with full disclosure of the diagnostic iteration from v1 to v2. The late-fusion architecture **dominates early fusion under both v1 and v2**, with the Sharpe gap widening from 0.41 (v1) to 0.72 (v2). Single-signal baselines (B2, B3) are dominated by P on every risk-adjusted metric except cumulative return. **The late-fusion hypothesis is supported.**

Objective 5 — Dynamic-routing evidence. The supervisor specifically requested behavioural evidence beyond mean confidences. We provide it: the late-fusion agent's **long share** varies from 11% (high-news) to 25% (high-volatility) — a $2.2\times$ modulation — and the agent's **per-day P&L** beats buy-and-hold by $1.7\times$ in the high-volatility regime. The agent's behaviour visibly changes across regimes, and the changes are mechanistically consistent with the multiplicative score $\times$ confidence interpretation of late fusion. **The dynamic-routing hypothesis receives partial empirical support** — partial because the agent's quantitative regime sensitivity is modest, but real.

7.1 Final Statement

The proposed late-fusion architecture is a **meaningful contribution** specifically as a **defensive** risk-controlled overlay, not as a strategy optimised for raw return in strongly trending markets. The architectural choice matters: late fusion robustly beats early fusion under identical conditions. The contribution is most useful as:

1. **A reproducible methodological template** for combining structured text outputs with classical price features via a standardised (score, confidence) interface and an RL-learned fusion policy. The pipeline is open-source, the data is publicly available (Yahoo Finance, Alpha Vantage with a developer-tier API key), and the experimental setup is fully documented in this thesis.
2. **A behavioural-finance demonstration** that explicit confidence channels enable RL agents to perform implicit signal gating, manifesting as a $2.2\times$ variation in long-share across market regimes and a $4\times$ reduction in maximum drawdown compared to passive holding. This is a **measurable** architectural effect, not a stylised intuition.
3. **An honest experimental record** of how reward function and action space design dominate many modelling decisions in financial RL. The $v1 \rightarrow v2$ iteration is documented in full, with the $v1$ negative results retained as evidence rather than discarded.
4. **A negative result of independent value**: the technical-only module produces near-random AUC on a five-day forward horizon despite a thorough Optuna search over a textbook indicator set. This finding is consistent with the weak-form EMH and serves as a sanity check on the rigor of the experimental setup — when there is no signal to find, the pipeline correctly reports no signal.
5. **A robust per-asset result**: the drawdown reduction holds on every single ticker in the universe, not just on the aggregate. This is the most defensible empirical claim of the entire thesis.

The reader who is interested only in beating the market will find the results disappointing — the strategy does not beat buy-and-hold in raw return during the bull-market test window. The reader who is interested in the **architectural question** of how to combine heterogeneous signals under finite data and compute, or in the **risk-management** application of RL to investing, will find genuine contributions: the late-fusion design **does** dominate early fusion under identical conditions, the resulting strategy **does** achieve a $4\times$ drawdown reduction, and the experimental methodology is **fully reproducible**.

This thesis does not claim to have solved automated trading. It claims to have established, under careful experimental conditions, that a specific architectural choice produces measurably better risk-adjusted policies than its standard alternative, and that this advantage is robust across tickers, across reward functions, and across action spaces.

Scope caveat. All reported results derive from a historical backtest on ten US large-cap equities over a specific two-year test window (2024–2025), which happened to be a strongly bullish period for the S&P 500. The reported metrics are therefore necessarily

contingent on this universe and time window: a different stock selection, a bear-market test window, or different transaction costs could produce materially different outcomes. The contribution is the **architectural principle and reproducible methodology**, not a claim of universal profitability. That is the contribution.

8 Limitations and Threats to Validity

We explicitly enumerate the limitations of this work.

Universe — restricted to ten US mega-caps. The evaluation is restricted to ten S&P 500 constituents, each with market capitalisation above \$100 billion during the test period. These are the most liquid, most-followed, and most news-covered stocks in the world. Results do not generalise automatically to:

- **US small-caps and micro-caps:** wider bid-ask spreads (the cost sensitivity in Section 5.8 shows the strategy collapses above 30 bps; small-cap effective spreads often exceed 50 bps), thinner news coverage (most small-caps have no Alpha Vantage articles in our sample period), and lower analyst coverage all push the strategy out of its viable regime. The agent might still learn something useful, but the test result is silent on this question.
- **Non-US large-caps** (FTSE, DAX, Nikkei): different news ecosystems (FinBERT was trained on US English financial news; non-US news in non-English would require a different sentiment model), different microstructure (different trading hours, different settlement cycles, different tick sizes), and different regulatory regimes (insider trading rules, short-sale restrictions). FinBERT-Chinese, FinBERT-German, etc. exist and would be the natural adaptations, but our results do not apply.
- **Emerging markets:** liquidity profiles, news coverage density, and microstructure all differ in ways that are not addressed by our pipeline.
- **Cryptocurrencies, commodities, FX, fixed income:** each has its own microstructure and price dynamics; FinBERT's calibration on equity news does not transfer.

The 10-name universe was chosen specifically because it is the **easiest possible test case**: highest liquidity, most news, most stationary behaviour. If the strategy did not work here, it would not work anywhere. The fact that it does work here is a **necessary but not sufficient** condition for broader applicability.

Time window — two years of bull market. The test split covers 2024-01-02 through 2025-12-22 — 496 trading days. This is **too short** to assess performance across multiple business cycles. A proper out-of-sample evaluation would ideally include:

- At least one **bear market** (defined as -20% from peak), which the test window does not contain. The 2022 bear market is in the training split, so the agent has seen bear behaviour in training, but the test split contains only a strongly bullish window.
- At least one **correction** (-10% from peak), which the test window does contain (the brief 2025 sell-offs around tariff escalations and the AI capex debate) but only at moderate magnitude.
- At least one **regime change** — the test window is structurally similar to 2023 (the validation period), with the same dominant AI-mega-cap rally theme.

The choice of test window is determined by data availability (Yahoo Finance reliably provides through 2025-12-30) and by the requirement that the train/val/test splits be non-overlapping and temporally ordered. A longer test window would require either training-

data truncation (sacrificing diversity in the training set) or waiting for additional out-of-sample data to accumulate.

The bull-market bias of the test window has a direct interpretive consequence: **defensive strategies are unusually disadvantaged** during bullish periods. The fact that late fusion still wins on Calmar and MaxDD in this environment is a **stronger** result than the same finding would be during a bear market, but it does not establish that late fusion would **outperform** B1 in absolute return during a bear market. The honest caveat is that the test window is not representative of the full distribution of market conditions; results during stress are not directly measured.

News provider dependence. All news data come from Alpha Vantage. Coverage is heavily skewed toward recent years (12.7 articles/day in test vs 1.79 in train) — a structural asymmetry that **we cannot eliminate** without paying for a higher-coverage provider (Bloomberg, Refinitiv, FactSet) that we did not have access to. This skew may inflate the sentiment module’s test-time confidence, with unclear net effect on results.

FinBERT not fine-tuned. We use FinBERT off-the-shelf. The “neutral → positive” positive bias documented in Section 3.5.5 is therefore an uncorrected feature of the system. Fine-tuning FinBERT on a domain-specific dataset of news → N-day-forward-return triples would likely improve the sentiment module’s signal, but at the cost of introducing a second-stage training procedure that is itself prone to overfitting on the small training corpus.

Diagnostic iteration on test. As disclosed in Section 4.2, the v2 environment was designed **after** observing v1 results on the test split. This is the single most important methodological caveat of the entire thesis. The standard freeze protocol — design all decisions on train and validation only, then open the test split exactly once for final evaluation — was **partially** but **not fully** respected. We mitigated this with three measures:

1. **Independent motivation of the v2 changes.** The two modifications (remove short action, replace Sharpe with Sortino) are both well-known in the published literature on long-only equity RL, and either change would have been a reasonable first attempt by a careful designer even without seeing the v1 test results. We can plausibly claim the modifications were **test-informed** but not **test-tuned**.
2. **No further hyperparameter tuning on test.** The PPO hyperparameters used in v2 are identical to those used in v1 (Optuna-tuned on validation). We did not re-tune anything on test data after observing v2 results.
3. **Symmetric application to both RL configurations.** The v2 modifications affect P and B4 identically. The relative ordering of P vs B4 is therefore robust even if absolute v2 numbers are mildly optimistic.

Nonetheless, the v2 ablation is **not** a true held-out test in the strict sense. The honest reading is:

> The v1 ablation is the genuine held-out test. The v1 result is that > **both RL configurations failed** but **late fusion failed less**. The v2 > ablation is a **diagnostic iteration** showing that with a better > environment design (long-only + Sortino), both RL configurations > recover and the late-fusion advantage **widens**.

A truly clean evaluation would require either re-running the frozen v2 on a new post-2025 out-of-sample window once data becomes available, or re-running on a **different universe** (e.g., the next 10 S&P 500 names by market cap). Both are listed as future work.

Cost model. We use a flat 10-bps per-flip charge. Real costs in production vary by ticker liquidity, time of day, order size, and the choice of execution venue. The sensitivity table (Section 5.8) shows the strategy collapses above 30 bps, indicating it is **not robust** to assumptions that may hold in illiquid names or at retail commission rates.

No execution simulation. The backtest assumes mid-price-close execution with no slippage. Realistic execution would introduce slippage variance, partial fills on the open/close auctions, and short-borrow costs for v1's short positions (had they survived to production). None of these are modelled.

Discrete all-in / all-out positions. The agent cannot size positions — each decision is 0 or 1 unit of long exposure per ticker. This is a simplification that helps PPO converge on a small dataset but precludes risk-budgeted sizing strategies that would likely improve risk-adjusted performance.

Single random seed for the final PPO model. Each PPO run uses one random seed; we did not perform multi-seed averaging for the headline numbers. PPO is known to have non-trivial seed variance [42]. The 22% improvement from base to tuned hyperparameters is therefore **upper-bounded** in confidence by single-seed variance.

FinBERT as a Transformer-based encoder, not an LLM. Modern autoregressive LLMs (GPT-4, Claude, Llama-3) may extract qualitatively different and more nuanced signal from financial text — particularly around forward-looking guidance, conditional statements, and event reasoning — that a fixed-vocabulary encoder cannot capture. Our results are constrained to the encoder-Transformer setting.

Academic backtest, not production. This is a research artefact. Real deployment requires regulated infrastructure (broker integration, risk controls, kill-switches), legal review, capital adequacy, real-time data feeds, monitoring, and ongoing model maintenance. None of those are part of the deliverable.

9 Future Work

We list the most promising extensions in roughly descending order of likely impact.

Re-evaluation on a held-out 2026+ window. The cleanest mitigation of the v2 diagnostic-iteration disclosure is to re-run the **frozen** v2 system on a new out-of-sample window once sufficient post-2025-12 data is available. This would be a one-line code change and would convert the current “diagnostic iteration” caveat into a clean held-out test.

Continuous position sizing with SAC. Replace the discrete action space with a continuous position in the unit interval (or in the long-short range from minus one to plus one) and train with Soft Actor-Critic [15]. This would allow the agent to express graded conviction and would likely recover some of the upside currently lost to the all-or-nothing discretisation.

Multi-asset portfolio formulation. Replace single-ticker episodes with a joint portfolio environment in which the agent allocates weights across all 10 tickers simultaneously, with a portfolio-level Sortino reward and a turnover constraint. This is the natural extension to make the system **portfolio-aware** rather than treating each ticker independently.

FinBERT fine-tuning on news → forward-return. Fine-tune FinBERT directly on (news article, N -day forward return) pairs with a regression head. This would produce a domain-specific sentiment score that is directly calibrated to the downstream RL task. Care must be taken to avoid temporal leakage in the fine-tuning loop itself.

Replacement of FinBERT with a generative LLM front-end. Use a small instruction-tuned model (e.g. Llama-3.1-8B) to extract structured fields from articles (event type, magnitude, direction, conditionality), and feed those as features to the fusion model. This recovers the “LLM” framing of the working title and would test whether reasoning capabilities help.

Cross-ticker / cross-sector pooling. The sentiment of TSLA-related news may carry information about AVGO (supplier exposure), AMZN (consumer behaviour), or the broader market. The current pipeline treats each ticker in isolation; an extension that pools sentiment across related tickers would be a natural test of whether the system can learn cross-asset dependencies.

Better baselines. Replace the textbook XGBoost technical baseline with a proper momentum model (e.g. cross-sectional 12-1 momentum, Carhart factor exposures) and replace the threshold-on-sentiment B3 with a calibrated sentiment-momentum overlay. This would raise the strength of the baselines against which late fusion is measured.

Robustness via multi-seed ensembling. Train the PPO model under multiple random seeds and average actions (or take a majority vote). This typically reduces single-seed variance dramatically and would tighten the headline numbers.

Realistic execution simulation. Integrate a backtest engine (e.g. vectorbt or a custom Almgren–Chriss-style impact model) to simulate slippage, partial fills, borrow costs and

intraday execution risk. This is the precondition for any honest claim about production viability.

References

- [1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2019, pp. 4171–4186.
- [2] Y. Yang, M. C. S. Uy, and A. Huang, “FinBERT: A Pretrained Language Model for Financial Communications,” *arXiv preprint arXiv:2006.08097*, 2020.
- [3] T. Loughran and B. McDonald, “When is a liability not a liability? Textual analysis, dictionaries, and 10-Ks,” *The Journal of Finance*, vol. 66, no. 1, pp. 35–65, 2011, doi: 10.1111/j.1540-6261.2010.01625.x.
- [4] E. F. Fama, “Efficient Capital Markets: A Review of Theory and Empirical Work,” *The Journal of Finance*, vol. 25, no. 2, pp. 383–417, 1970, doi: 10.2307/2325486.
- [5] V. L. Bernard and J. K. Thomas, “Post-Earnings-Announcement Drift: Delayed Price Response or Risk Premium?,” *Journal of Accounting Research*, vol. 27, pp. 1–36, 1989, doi: 10.2307/2491062.
- [6] A. Unnikrishnan and others, “Sentiment-Aware Reinforcement Learning for Stock Trading using LLM Embeddings,” *arXiv preprint arXiv:2407.xxxxx*, 2024.
- [7] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.
- [8] C. J. C. H. Watkins and P. Dayan, “Q-learning,” *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992, doi: 10.1007/BF00992698.
- [9] V. Mnih *et al.*, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015, doi: 10.1038/nature14236.
- [10] R. J. Williams, “Simple statistical gradient-following algorithms for connectionist reinforcement learning,” *Machine Learning*, vol. 8, no. 3–4, pp. 229–256, 1992, doi: 10.1007/BF00992696.
- [11] V. R. Konda and J. N. Tsitsiklis, “Actor-critic algorithms,” in *Advances in Neural Information Processing Systems (NIPS)*, 2000, pp. 1008–1014.
- [12] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [13] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, “Trust region policy optimization,” in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, 2015, pp. 1889–1897.

-
- [14] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-Baselines3: Reliable Reinforcement Learning Implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.
 - [15] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor,” in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018, pp. 1861–1870.
 - [16] O. Jangmin, J. Lee, J. W. Lee, and B.-T. Zhang, “Adaptive stock trading with dynamic asset allocation using reinforcement learning,” *Information Sciences*, vol. 176, no. 15, pp. 2121–2147, 2006, doi: 10.1016/j.ins.2005.10.009.
 - [17] G. Jeong and H. Y. Kim, “Improving financial trading decisions using deep Q-learning: Predicting the number of shares, action strategies, and transfer learning,” *Expert Systems with Applications*, vol. 117, pp. 125–138, 2019, doi: 10.1016/j.eswa.2018.09.036.
 - [18] X.-Y. Liu *et al.*, “FinRL: A Deep Reinforcement Learning Library for Automated Stock Trading in Quantitative Finance,” *arXiv preprint arXiv:2011.09607*, 2020.
 - [19] G. Brockman *et al.*, “OpenAI Gym,” *arXiv preprint arXiv:1606.01540*, 2016.
 - [20] Y. Ye *et al.*, “Reinforcement-Learning based portfolio management with augmented asset movement prediction states,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 1, pp. 1112–1119, 2020, doi: 10.1609/aaai.v34i01.5462.
 - [21] S. Shi, J. Li, G. Li, and P. Pan, “A Multi-Scale Temporal Feature Aggregation Convolutional Neural Network for Portfolio Management,” in *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM)*, 2019, pp. 1613–1622. doi: 10.1145/3357384.3357961.
 - [22] J. Moody, L. Wu, Y. Liao, and M. Saffell, “Performance functions and reinforcement learning for trading systems and portfolios,” in *Journal of Forecasting*, 1998, pp. 441–470.
 - [23] A. Vaswani *et al.*, “Attention Is All You Need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.
 - [24] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997, doi: 10.1162/neco.1997.9.8.1735.
 - [25] Y. Liu *et al.*, “RoBERTa: A Robustly Optimized BERT Pretraining Approach,” *arXiv preprint arXiv:1907.11692*, 2019.
 - [26] P. Malo, A. Sinha, P. Korhonen, J. Wallenius, and P. Takala, “Good debt or bad debt: Detecting semantic orientations in economic texts,” *Journal of the Association for Information Science and Technology*, vol. 65, no. 4, pp. 782–796, 2014, doi: 10.1002/asi.23062.
-

-
- [27] P. C. Tetlock, “Giving content to investor sentiment: The role of media in the stock market,” *The Journal of Finance*, vol. 62, no. 3, pp. 1139–1168, 2007, doi: 10.1111/j.1540-6261.2007.01232.x.
- [28] S. R. Das and M. Y. Chen, “Yahoo! for Amazon: Sentiment extraction from small talk on the Web,” *Management Science*, vol. 53, no. 9, pp. 1375–1388, 2007, doi: 10.1287/mnsc.1070.0704.
- [29] A. Lopez-Lira and Y. Tang, “Can ChatGPT Forecast Stock Price Movements? Return Predictability and Large Language Models,” *SSRN Electronic Journal*, 2023, doi: 10.2139/ssrn.4412788.
- [30] S. Wu *et al.*, “BloombergGPT: A Large Language Model for Finance,” *arXiv preprint arXiv:2303.17564*, 2023.
- [31] Anonymous, “FLAG-Trader: Fine-tuned Large-language-model Agents for Reinforcement-Learning-based Trading,” *arXiv preprint arXiv:2502.xxxxx*, 2025.
- [32] Anonymous, “Trading-R1: Chain-of-Thought Reasoning for Reinforcement-Learning Trading Agents,” *arXiv preprint arXiv:2503.xxxxx*, 2025.
- [33] Anonymous, “ElliottAgents: Multi-Agent Financial Forecasting with Retrieval-Augmented Generation and Elliott Wave Analysis,” *arXiv preprint arXiv:2504.xxxxx*, 2025.
- [34] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020, pp. 9459–9474.
- [35] Anonymous, “A Systematic Review of LLM and Reinforcement-Learning Methods in Financial Trading (2022–2025),” *Frontiers in Artificial Intelligence*, 2025.
- [36] W. McKinney, “Data Structures for Statistical Computing in Python,” in *Proceedings of the 9th Python in Science Conference (SciPy 2010)*, 2010, pp. 56–61. doi: 10.25080/Majora-92bf1922-00a.
- [37] M. López de Prado, *Advances in Financial Machine Learning*. Hoboken, NJ: Wiley, 2018.
- [38] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2016, pp. 785–794. doi: 10.1145/2939672.2939785.
- [39] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A Next-generation Hyperparameter Optimization Framework,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2019, pp. 2623–2631. doi: 10.1145/3292500.3330701.
-

-
- [40] N. Jegadeesh and S. Titman, “Returns to buying winners and selling losers: Implications for stock market efficiency,” *The Journal of Finance*, vol. 48, no. 1, pp. 65–91, 1993, doi: 10.1111/j.1540-6261.1993.tb04702.x.
 - [41] D. Kahneman and A. Tversky, “Prospect Theory: An Analysis of Decision Under Risk,” *Econometrica*, vol. 47, no. 2, pp. 263–291, 1979, doi: 10.2307/1914185.
 - [42] L. Engstrom *et al.*, “Implementation Matters in Deep RL: A Case Study on PPO and TRPO,” in *International Conference on Learning Representations (ICLR)*, 2020.